

PCNN_{CEC}: Efficient and Privacy-Preserving Convolutional Neural Network Inference Based on Cloud-Edge-Client Collaboration

Jing Wang, Debiao He^{ID}, *Member, IEEE*, Aniello Castiglione^{ID}, *Member, IEEE*, Brij B. Gupta^{ID}, *Senior Member, IEEE*, Marimuthu Karupiah^{ID}, *Senior Member, IEEE*, and Libing Wu^{ID}

Abstract—Deploying convolutional neural network (CNN) inference on resource-constrained devices remains a remarkable challenge for Industrial Internet of Things (IIoT). Although the cloud computing shows great promise in machine learning training and prediction, outsourcing data to a remote cloud always incurs privacy risk and high latency. Therefore, we design a new framework for efficient and privacy-preserving CNN inference based on cloud-edge-client collaboration (named PCNN_{CEC}). In PCNN_{CEC}, the model of cloud and the data of client in IIoT are split into two secret shares and sent to two non-colluded edge servers. We proposed a new efficient private comparison protocol based on the additively secret sharing technique, which can be used to realize secure computation of ReLU function without approximation in semi-honest adversary model. By applying some secure two-party computation protocols,

the two edge servers can jointly calculate the predicting results without learning anything about the model and data. Moreover, to speed up the pre-computation of offline phase but not sacrifice security, we delegate the task of triplets generation to the cloud, so that the edge servers do not require frequent interactions to generate triplets themselves or introducing additional trusted party. The experimental results show the proposed private comparison protocol achieves a better tradeoff between low latency and high throughput, when it is compared with garbled circuit based protocols and other secret sharing based protocols. Additionally, the benchmarks conducted on realistic MNIST and CIFAR-10 datasets demonstrate that PCNN_{CEC} costs less communication and runtime than two recently related schemes under the same security level.

Index Terms—Convolutional neural network, edge computing, industrial Internet of Things, privacy-preserving inference.

Manuscript received 16 November 2021; revised 12 April 2022; accepted 22 May 2022. Date of publication 26 May 2022; date of current version 20 September 2023. The work was supported in part by the National Natural Science Foundation of China under Grants 61972294, 61932016, and 62172307, in part by the Blockchain Core Technology Strategic Research Program of Ministry of Education of China under Grant 2020KJ010301, in part by the Special Project on Science and Technology Program of Hubei Province under Grant 2020AEA013, in part by the Natural Science Foundation of Hubei Province under Grant 2020CFA052, and in part by Wuhan Municipal Science and Technology Project under Grant 2020010601012187. Recommended for acceptance by Dr. Varun G Menon. (Corresponding authors: Debiao He; Brij B. Gupta.)

Jing Wang is with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, Hubei 430074, China, and also with the Shanghai Key Laboratory of Privacy-Preserving Computation, MatrixElements Technologies, Shanghai 201204, China (e-mail: cswjing@hust.edu.cn).

Debiao He is with the Key Laboratory of Cryptography of Zhejiang Province, Hangzhou Normal University, Hangzhou, Zhejiang 311121, China, with the Shandong Provincial Key Laboratory of Computer Networks, Qilu University of Technology (Shandong Academy of Sciences), Jinan, Shandong 250014, China, and also with the School of Cyber Science and Engineering, Wuhan University, Wuhan, Hubei 430072, China (e-mail: hedebiao@163.com).

Aniello Castiglione is with the Department of Science and Technology, University of Naples Parthenope, I-80123 Naples SA, Italy (e-mail: castiglione@ieec.org).

Brij B. Gupta is with the International Center for AI & Cyber Security Research & Innovations, and the Department of Computer Science and Information Engineering, Asia University, Taichung 413, Taiwan, with Lebanese American University, Beirut 13-5053, Lebanon, and also with the Department of Computer Science, King Abdulaziz University, Jeddah 21589, Saudi Arabia (e-mail: bbgupta@nitkkr.ac.in).

Marimuthu Karupiah is with the Department of Computer Science and Engineering, SRM Institute of Science and Technology, Ghaziabad, Uttar Pradesh 201204, India (e-mail: marimuthume@gmail.com).

Libing Wu is with the School of Cyber Science and Engineering, Wuhan University, Wuhan, Hubei 430072, China (e-mail: whuwlb@126.com).

Digital Object Identifier 10.1109/TNSE.2022.3177755

I. INTRODUCTION

IN THE last decades, the rapid development of wireless communication technology has paved the technical way for the wide application of the Internet of Things (IoT). Nowadays, the IoT has been widely deployed in many application scenarios, such as intelligent transportation, home automation, smart healthcare, and even more important industrial fields. By integrating the IoT technology, those industrial systems can achieve real-time online sensor monitoring and more efficient control, thereby effectively reducing costs, improving efficiency and bringing revolutionary changes to the industrial development. Consequently, the concept of Industrial Internet of Things (IIoT) has been emerged. By collecting, analyzing industrial data and continuously self-optimizing for model building, IIoT improves the capabilities of decision-making and control to achieve system-level efficiency optimization [1], [2]. This means that machine learning becomes increasingly important for IIoT. However, there remains a question that how to deploy it.

Although the cloud has great advantages in data modeling and machine learning, it often cannot support real-time data collection or analysis required for those applications in manufacturing, transportation and smart building industries, etc. Therefore, there is a trend to enhance cloud capabilities with the help of edge computing [3], [4]. *Once the model is well-trained at the remote cloud, it then can be pushed to the edge.* When the data analytics move to the edge, the costs for

data storage and processing can be significantly reduced since the amount of data across the network is dramatically decreased. Moreover, the edge-ified machine learning can launch quick response to the anomalies detected in industrial manufacturing, and then help the manufacturers to improve productivity and reduce downtime. According to the report of Garner in 2018, there would be 75% of enterprise-generated data being processed at the edge within next four years [5]–[7].

Convolutional neural network (CNN) as a kind of deep learning algorithm, is proved to have great advantages in image and video recognition, and thus have been widely applied in IIoT [8]–[12]. For example, Wang *et al.* [13] applied CNN in visual sorting systems for flexible manufacturing systems based on the cloud-edge computing architecture. However, outsourcing models and data to a not fully trusted edge server raises serious privacy and security concerns. An adversary can intercept IIoT data at the client and then launch linkage attack to obtain other sensitive information [14]; On the other hand, the adversary may steal the model via inference of machine learning for commercial profits since the model is a valuable asset [15], [16]. To facilitate the applications based on machine-learning-as-a-service, there have been many schemes being proposed to realize privacy-preserving inferences [17]–[21]. Considering a service provider holds a model $\mathbf{W}$ of a CNN algorithm f and a client holds its private data $\mathbf{x}$, the goal of the privacy-preserving inference is to compute the prediction $f(\mathbf{W}, \mathbf{x})$, on the provided that service provider cannot learn anything about private $\mathbf{x}$ and the client can get nothing about the model $\mathbf{W}$ either.

To achieve the above goal, an intuitive solution is to use the homomorphic encryption (HE) cryptosystem, which allows linear computations on ciphertext. In this way, models and industrial data can be encrypted before being outsourced to an edge server for privacy preservation. However, HE does not support nonlinear computations, so that some HE-based schemes have to transfer the non-linear functions into approximately degree polynomials, which may reduce the accuracy of predictions [22]. Another solution is to utilize secure multi-party computation (SMPC) technology. In SMPC-based schemes, there are multiple entities for jointly computing the function $f(\mathbf{W}, \mathbf{x})$, but each of them only have a share of $\mathbf{W}$ and $\mathbf{x}$ that they can learn nothing about $\mathbf{W}$ and $\mathbf{x}$. Furthermore, SMPC-based schemes can be divided into two categories [20]: The first one is based on the garbled circuit (GC) technique [23] for low latency, which results in constant rounds but high communication overheads; the other one is based on secret sharing (SS) technique for high throughput but more interactive rounds than GC-based category.

To obtain a tradeoff between latency and throughput, some researchers proposed mixed protocol framework [24]–[26], where linear functions are calculated based on SS and nonlinear functions (e.g., ReLU activations in CNN) are calculated by using GC. But the conversions between arithmetic secret shares and GC shares incur additional communication costs and rounds. Some schemes

that perform nonlinear functions efficiently require sacrificing security by assuming the existence of a trusted third party to participant the protocol executions or by leaking the activations [20]. Therefore, effective and efficient privacy-preserving inference over realistic neural network still remains elusive.

A. Our Contributions

Motivated by achieving a balance among security, accuracy, low latency and high throughput, this paper presents a new privacy-preserving convolutional neural network inference based on cloud-edge-client collaboration (PCNN_{CEC}) for IIoT. In our scheme, the cloud represents a service provider that holds models and provides prediction services, and the private predicting computation is performed by two non-collocated edge servers. The main contribution of this paper is summarized as follows.

- First, we design a new private inference based on the cloud-edge-client framework. By applying the lightweight arithmetic secret sharing (ASS) with beaver's triplets [27], the computation-intensive online prediction is moved to two edge servers, and the task of offline triplets generation is delegate to the service provider (i.e., cloud). Compared with existing schemes, we do not introduce additional trusted party, and speed up the offline computations by avoiding the interaction for triplets generation between two edge servers.
- Second, we propose a new efficient two-party protocol $\Pi_{\text{privComp}}^{2P}$ for privacy-preserving comparison operation. The simplified version can be used to realize private the ReLU function and maximum pooling computation in CNN without making any approximation, so that the proposed PCNN_{CEC} can achieve the same accuracy with the centralized inference. While the complete version can be used to realize the derivative of a ReLU function (DReLU) in CNN training. The experimental results show that $\Pi_{\text{privComp}}^{2P}$ shows obtains a good trade-off between low latency and high throughput when it is run in parallel with batch size larger than 100, especially in the WAN setting.
- We give concrete security analysis to prove the privacy property of PCNN_{CEC}. And we conduct a full implementation of PCNN_{CEC} on two realistic datasets (i.e., MNIST, CIFAR-10) to evaluate its performance. The experimental results demonstrate that the proposed PCNN_{CEC} is more efficient than previous works on both MNIST and CIFAR-10 datasets.

B. Organization of the Paper

The reminder of this paper is outlined as follows. Section II reviews some previous schemes about the privacy-preserving inference, and Section III gives the preliminaries and problem statements for the proposed framework. Section IV presents some building blocks related to the proposed scheme PCNN_{CEC}, including our proposed privacy-preserving comparison protocol $\Pi_{\text{privComp}}^{2P}$. Then, Section V describes the

details of PCNN_{CEC}. And a formal security proof of the proposed protocol is demonstrated in Section VI. We then give a comparison on the performance and security with some related protocols to demonstrate the practicability of PCNN_{CEC} in Section VII. Finally, a conclusion is drawn in the last section.

II. RELATED LITERATURES

Machine learning (ML) services based on the deep neural networks have shown remarkable potentials in predictions and classifications, and have been widely used in many fields, including industrial IoT. When comes to the machine learning, it is important to solve the privacy problem, because traditional ML inference should access the raw data that is sensitive and may bring potential privacy risks [28], [29]. For this, a number of schemes for privacy-preserving inference of ML have been proposed.

Taking the advantages of computing any polynomial operations on encrypted data, fully homomorphic encryption (FHE) is widely applied in privacy-preserving machine learning inference, especially for linear regression, decision-making tree, support vector machine and Naïve Bayes classification [30]–[32]. CryptoNets proposed by Microsoft was the first to consider the privacy-preserving neural network inference based on FHE, but it cannot directly execute the non-polynomial activated operations (e.g., sigmod function). To address the problem, CryptoDL and E2DM attempted to replace these non-polynomial functions with approximately polynomials of high degrees. However, it still remains a challenge to the computation-constrained clients.

To reduce the computational costs, some works exploited to replace the FHE with partial homomorphic encryption, and make an assumption that there are two non-colluded servers for private training and predictions [33]–[35]. However, the limitation on computing nonlinear functions still cannot be tackled, since the approximation polynomials may degrade the inference accuracy. Fortunately, when the assumption of existing two or more non-colluded servers meets the secure multi-party computation, the accuracy and computational efficiency gets a significant improvement.

SecureML proposed by Mohassel and Zhang [36] was the first to provide a general machine learning model training method for data privacy protection. In their scheme, the training data was shared to two non-colluding servers based on ASS. The model training process for linear regression network and simple neural network is realized by applying classical secure two-party computation technologies, such as GC, oblivious transfer (OT) [37] and beaver triplets. Demmler *et al.* proposed a mixed-protocol framework (called ABY) [24], which combined the arithmetic secret sharing, boolean secret sharing and Yao's garbled circuit to realize secure two-party linear and nonlinear operations, and instantiated them in general machine learning models. The experimental results show that ABY is more efficient than SecureML. Patra *et al.* [25] improved the efficiency of ABY and proposed a new mixed-protocol framework-ABY2.0. Mohassel *et al.*

TABLE I
NOTATIONS

Notations	Illustrations
$\mathcal{SP}$	Service provider
$\mathcal{ES}_i$	The i -th edge server, where $i \in \{1, 2\}$
$\mathcal{DV}$	IoT device
$[x]$	Boolean additive share for secret x
$[[x]]$	Arithmetically additive share for secret x
$ x $	Absolute value of x
$\cdot$	Inner product operation
$\times$	(Element-wise) multiplication operation for arithmetic value
$+$	(Element-wise) addition operation for arithmetic value
$-$	(Element-wise) subtraction operation for arithmetic value
$\otimes$	AND-operation for boolean value
$\oplus$	XOR-operation for boolean value

extended the secure two-party framework to a three-party framework-ABY3 [26], and proposed a more flexible secure data truncation protocol applicable to more than three parties, which can be better applied to the realistic model training and prediction schemes. Inherited by the ideas of above works, many other privacy-preserving machine learning schemes have been proposed.

Liu *et al.* [38] proposed a secure two-party based oblivious neural network prediction scheme, named MiniONN. Yet it is proved that the efficiency of this scheme has not been greatly improved, since it used the GC to realize nonlinear operations. Riazi *et al.* [18] proposed a new end-to-end architecture XONN based on GC, which replaced the matrix multiplication with the XNOR gate operation of GC, and provided a new customized neural network scheme to minimize the runtime of GC without sacrificing the prediction accuracy. Chandran *et al.* designed EzPC [39], the first secure two-party computing protocol compiler combining arithmetic and boolean circuits, which can be used to implement privacy-preserving computations based on the two circuits. However, Li *et al.* [21] pointed out that MiniON, XONN and EzPC cannot preserve model privacy. Moreover, most of them use approximation polynomials to replace activation functions in neural networks or not provide a concrete protocol for the nonlinear computations. Rathee *et al.* [20] proposed a machine learning framework for secure two-party inference named CryptFlow2. They designed a new private ReLU protocol to improve the efficiency when compared with GC-based private ReLU protocols, but our experimental results showed that the protocol is not efficient enough. Thus, there is still a long way to design more practical solutions for privacy-preserving machine learning inference that can achieve a perfect balance between security and efficiency.

III. PRELIMINARIES AND PROBLEM STATEMENTS

Here, we first briefly review the convolutional neural network (CNN) and the additive secret sharing technique for privacy preservation used in the proposed system. Then, we introduce the problem statements about the proposed framework, i.e., system model, threat model and design goals.

A. Preliminaries

1) *Convolutional Neural Network*: CNN is essentially a sequence of network layers with multiple neurons. Each neuron is a nonlinear operation on a linear transformation of the previous layer's outputs. A typical CNN usually consists of five kinds of layers: input layer, convolutional layer, activation layer, pooling layer and fully-connected layer [40].

- 1) *Input layer*. Similar to the traditional neural network / machine learning, the input layer is the first layer of the model that is used to preprocess the input to improve the quality of training and prediction. Common data / image preprocessing methods include: mean taking, normalization, PCA / SVD dimension reduction, etc.
- 2) *Convolutional layer*. The convolutional layer is the core for a CNN. This layer is to compute the convolutions, essentially the linear transformations consisting of *inner productions* and *additions* between the predefined filter / kernel and the neighborhood of an element in the input volume. For example, give a kernel $\mathbf{w} = \{w_{r,c}\}$ of size $n \times n$, a neighborhood $\mathbf{x}_{j,i}$ of the input volume $\mathbf{X}$, the linear transformation result is represented as

$$net_{j,i} := \mathbf{w} \cdot \mathbf{x}_{j,i} + b = \sum_{r=0}^{n-1} \sum_{c=0}^{n-1} w_{r,c} x_{j+r,i+c} + b \quad (1)$$

where b is the bias bounded to $\mathbf{w}$. The linear computation is repeated, by sliding each kernel by a certain amount in every step. After several repetitions, the final output *net* of this layer is returned.

- 3) *Activation layer*. Element-wise nonlinear activation functions are usually applied after the convolutional layer or fully connected layer to model nonlinear relationships between inputs and outputs. Commonly used nonlinear activation functions in neural networks include sigmoid, tanh, softmax, ReLU, etc. Taking the advantage of overcoming the vanishing gradient problem and thus allowing models to learn faster and perform better, ReLU has been regarded as the default activation function in deep neural network. The piece-wise activation function ReLU is defined as

$$y := \sigma(x) = \max\{x, 0\} \quad (2)$$

- 4) *Pooling layer*. The pooling layer is applied to perform the downsampling operations by transforming the outputs of neuron clusters at the activation layer into a single neuron in the next layer, reducing the dimensions of feature maps. Max-pooling (denoted as MaxPool) and average-pooling (denoted as AvgPool) are two commonly used pooling methods. Specifically, MaxPool is to pick up the maximum value in a region of the feature map covered by the kernel, while AvgPool is to compute the average value in a region of the feature map covered by the kernel.

- 5) *Fully connected layer*. In the CNN, the fully connected layer are usually deployed in the last few layers, compiling the features extracted by previous layers to form the final output. Each neuron in this layer has full connections to all neurons in previous layer. Given an input $\mathbf{X} = \{\mathbf{x}_0 \cdots \mathbf{x}_{d-1}\}$ and a weight matrix $\mathbf{W} = \{\mathbf{w}_0 \cdots \mathbf{w}_{d-1}\}$ for the fully connected layer, the output can be computed as

$$z_i = \mathbf{w}_i \cdot \mathbf{x}_i + b_i \quad \text{for } 0 \leq i \leq d-1 \quad (3)$$

where $\mathbf{X}$ is generated by transforming the output matrix in the previous activation layer into corresponding column vectors as the input of fully connected layer, d is the size of $\mathbf{W}$. The output $\mathbf{z} = \{z_i\}_{i=0}^{d-1}$ denotes the final classification vector if the current fully connected layer is the last layer.

In some works, the activation layer is often integrated into the convolutional layer. Thus, the structure of a general CNN structure can be formally expressed as the pattern: $\text{Input} \rightarrow [\text{Conv} \rightarrow \text{ReLU}] \times N \rightarrow \text{Pool?} \times M \rightarrow [\text{FC} \rightarrow \text{ReLU}] \times K \rightarrow \text{FC}$, where Input, Conv, ReLU, Pool and FC denote the above five types of layers respectively. The notation " $\times N$ ($/M/K$)" represents that the number of repetitions for the layers is N ($/M/K$). The notation "Pool?" denotes that the pooling layer can be a optional layer in the sub-structure.

2) *Additive Secret Sharing*: Additive secret sharing is a popular technique for efficient *secure multiparty computation* [41]. In this paper, we use (2, 2)-(additive) secret sharing, where two shares held by two parties add up to a secret value in a certain field. The additive secret sharing can be further divided into *arithmetic sharing* and *boolean sharing*.

Arithmetic secret sharing. Considering a secret to be shared between two parties is $x \in \mathbb{Z}_{2^l}$, one can generate two shares for the secret as $r \in \mathbb{Z}_{2^l}$ and $x - r \pmod{2^l}$ respectively, where r is randomly pick up from the ring $\mathbb{Z}_{2^l}$. At the view of one party, the share r is uniformly at random, so that it is distribution indistinguishable from any element in $\mathbb{Z}_{2^l}$. As the value r is at random, the share $(x - r)$ alone is also a random value at the view of the other party. Therefore, the shares will not leak any information about the secret x to the parties when they only hold a part of shares.

Boolean secret sharing. Considering a secret to be shared between two parties is $f \in \{0, 1\}$, one can generate two shares for the secret as $f_1 \in \{0, 1\}$ and $(x \oplus f_1) \in \{0, 1\}$ respectively, where f_1 is a randomly chosen bit. Similar to the arithmetic secret sharing, the boolean shares will not leak the original secret value f at the view of the two parties.

Presentation and properties of secret-shared values. To distinguish the arithmetic sharing and boolean sharing, we use unified to denote the additive shares between two parties. Specifically, for $i \in \{1, 2\}$ in the two-party computation setting, let the notation $[x]_i$ denote the arithmetic share of the i -th party when $x \in \mathbb{Z}_{2^l}$ is a secret to be shared over the ring $\mathbb{Z}_{2^l}$; and let $[f]_i$ denote the boolean share of the i -th party if the secret f is a boolean value (i.e., $f \in \{0, 1\}$). Therefore, we

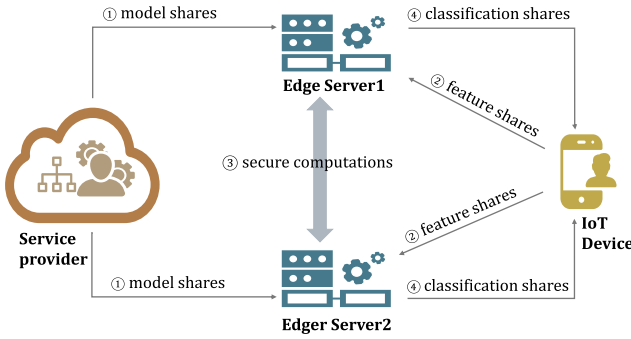


Fig. 1. System model.

can have

$$x = [x]_1 + [x]_2 \bmod 2^l \quad \text{and} \quad f = [f]_1 \oplus [f]_2$$

Given the shares $[u]_i$ and $[v]_i$ for secrets u, v and a constant value $c \in \mathbb{Z}_{2^l}$, it is obvious to see that the additive secret sharing satisfies the linear properties as follows:

$$[u]_i \pm [v]_i \bmod 2^l = [u \pm v]_i, \quad c \times [u]_i \bmod 2^l = [c \times u]_i$$

The linear properties are also suitable for the boolean shares $[u]_i, [v]_i$ and a boolean value $c \in \{0, 1\}$, such as

$$[u]_i \oplus [v]_i = [u \oplus v]_i, \quad c \otimes [u]_i = [c \otimes u]_i$$

For simplicity, we will omit the notation $\bmod 2^l$ for the arithmetic computations in the following text.

B. Problem Statement

1) *System Model*: To achieve an efficient and privacy-preserving CNN predictions for IoT infrastructure, our designed system involves three types of entities: server provider, edge server and IoT device, as illustrated in Fig. 1.

- *Service provider, SP*: The service provider, as the owner of an utility model, can be considered as a remote cloud that is far away from mobile clients. To relieve the burden and provide real-time services, it delegates most privacy-preserving computations to some region-orient edge servers. When receiving a service request, it only needs to secretly share its model parameters along with enough Beaver's triplet shares to two non-colluded edge servers in an element-wise manner.
- *IoT Devices, DV*: The IoT devices can be some Internet-connected sensors or other intelligent facilities that are resource-restrained in computation and bandwidth. It is hard for them to execute some secure but computation-intensive or bandwidth-consuming protocols with servers. Thus, to enjoy the service provided by SP without leaking personal data, an IoT device also generates additively secret shares of its private data in element-wise manner to two edge servers, and finally gets two shares that can add up to a final result.
- *Edge server(s), ES*: In the proposed system, there are two semi-honest and un-colluded edge servers that

connect the service provider and IoT devices. To protect the privacy of SP and DV, each of them can only get the shares of private model parameters and feature data, so that neither of them can learn SP's model parameters or DV's data. However, they can collaboratively run the private computations for privacy-preserving inference and return the shares of final result to the IoT device.

2) *Threat Model*: In this paper, we assume that the region-oriented edge servers have been authenticated with both the service provider (i.e., model owner) and the IoT devices (i.e., data owners), and they will be honest to execute all the secure two-party computations for the privacy-preserving CNN inference, but curious to the model parameters of service provider and feature data of IoT devices for some special purposes. And the two edge servers neither can collude with each other, nor collude with the service provider or the user. However, there are still two kinds of security threats. One is the leakage of user-data privacy: some adversaries (e.g., the service provider or the two edge servers) may attempt to infer the private data generated by user's IoT devices. The other one is the leakage of model privacy: some adversaries (e.g., the user or the edge servers) may try to learn the model parameters for personal interests.

3) *Design Goals*: Our goal is to design an efficient and privacy-preserving federal learning scheme for IoT environments with the following properties.

- *Correct*. The fundamental goal of the system is to correctly learn the prediction result from a pre-trained CNN model. The edge servers should collaboratively perform some two-party computations and generate correct prediction shares.
- *Privacy-preserving*. The important goal of the system is to protect the data privacy for IoT users and the model privacy for the services provider. During the two-party computation process, it should prevent the curious edge servers and other adversaries learning the model parameters and industrial data.
- *Efficient*. The computation and communication costs are the most important factors that affects the deployment of such system. It is necessary to reduce the computation and communication (i.e., bandwidth and rounds) overheads between entities.

IV. BUILDING BLOCKS

Here, we present some fundamental private computation sub-protocols used in PCNN_{CEC} framework. All these protocols are designed based on the (2,2)-additive secret sharing technique and will be run by ES₁ and ES₂.

A. Data Presentation

In this paper, we consider secure computations with three types of data form: binary (boolean) values $\mathbb{Z}_2 = \{f \in \{0, 1\} | f = 0 \text{ or } f = 1\}$, signed integers $\mathbb{Z}_{(l)} = \{v \in \mathbb{Z} | -2^{l-1} \leq v \leq 2^{l-1} - 1\}$ and fix-point rational numbers

$\mathbb{Q}_{(l,D)} = \{\tilde{v} \in \mathbb{Q} | \tilde{v} = v \times 2^{-lD}, v \in \mathbb{Z}_{(l)}\}$. Here, we can consider the signed integers are encoded in a ring $\mathbb{Z}_{2^l}$, and let the integer $\{v \in \mathbb{Z}_{2^l} | 2^{l-p-1} \leq v \leq 2^l - 1\}$ denote negative number and $\{v \in \mathbb{Z}_{2^l} | 0 < v < 2^{l-p-1}\}$ denote the positive number respectively, where $2^{lD} < 2^{lp} \ll 2^l$.

The computations with CNN inferences are executed on float-point numbers in reality, while secret sharing technique are usually works on integers that belongs to a ring (or field). Therefore, it is necessary to convert all float-point numbers into the “integers” in $\mathbb{Z}_{2^l}$. Similar to the solution in the literatures [21], [36], [38], we scale each float-point number up to an integer by multiplying a global constant and then dropping out the extra fractional parts. Specifically, assume that a float-point value to be scaled is $\bar{v} \in \mathbb{Q}$, it will be encoded as $v = \lfloor \bar{v} \times 2^{lD} \rfloor \in \mathbb{Z}_{2^l}$, where $\lfloor \cdot \rfloor$ denotes the round-down operation. While the data owner can recover the encoded value v as $\tilde{v} \in \mathbb{Q}_{(l,D)}$. In addition, to distinguish the positive values and negative ones, we must ensure that the absolute value of any “integers” (e.g., v) will not exceed $(2^{lp} - 1)$, and let $v \leq 2^{lp}$ denote the positive value, while the value v satisfying $2^{lp} < v < 2^l$ denotes the negative one.

B. Private Multiplication

Private multiplication protocol $\Pi_{\text{privMult}}^{2P}$ is the basic building block for our proposed system. Specifically, $\Pi_{\text{privMult}}^{2P}$ can be defined as follows: suppose there is a pair of shares $\{[x]_1, [y]_1\}$ and $\{[x]_2, [y]_2\}$ for $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively, then they can run the protocol to get $[z]_1$ and $[z]_2$ respectively, such that $[z]_1 + [z]_2 = ([x]_1 + [x]_2) \times ([y]_1 + [y]_2)$.

As the protocol $\Pi_{\text{privMult}}^{2P}$ will be frequently used in our proposed system, so its performance will directly influence the efficiency of the whole system. For this reason, it is significant to apply an efficient private multiplication protocol. Recently, Huang *et al.* [22] designed a novel private multiplication protocol by incorporating the idea of *Beaver's triplet* [27], and demonstrated that their protocol can greatly reduce the computation and bandwidth overheads, when compared with the protocols based on homomorphic encryption, garbled circuits and oblivious transfer techniques. Thus, we also apply their idea to achieve $\Pi_{\text{privMult}}^{2P}$. The Beaver's triplets used for $\Pi_{\text{privMult}}^{2P}$ would be in the form of $\{([\Delta a]_1, [\Delta b]_1, [\Delta c]_1), ([\Delta a]_2, [\Delta b]_2, [\Delta c]_2)\}$ such that $[\Delta c]_1 + [\Delta c]_2 = ([\Delta a]_1 + [\Delta a]_2) \times ([\Delta b]_1 + [\Delta b]_2)$, and it is usually preprepared ahead of time. Assume that the triplet shares $[\text{Tri}]_1 = \{[\Delta a]_1, [\Delta b]_1, [\Delta c]_1\}$ and $[\text{Tri}]_2 = \{[\Delta a]_2, [\Delta b]_2, [\Delta c]_2\}$ have been shared between the two parties $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively, then the protocol $\Pi_{\text{privMult}}^{2P}$ works as Fig. 2 shows.

Because the pair of Beaver's triplets (i.e., $\{[\text{Tri}]_1, [\text{Tri}]_2\}$) cannot be used repeatedly for security [27], we assume that every triplet used in the multiplication protocol is a new one. Moreover, as the triplet is a default input for each $\Pi_{\text{privMult}}^{2P}$, we thus *omit* the concrete notation of each triplet share in the description of following sub-protocols for simplicity.

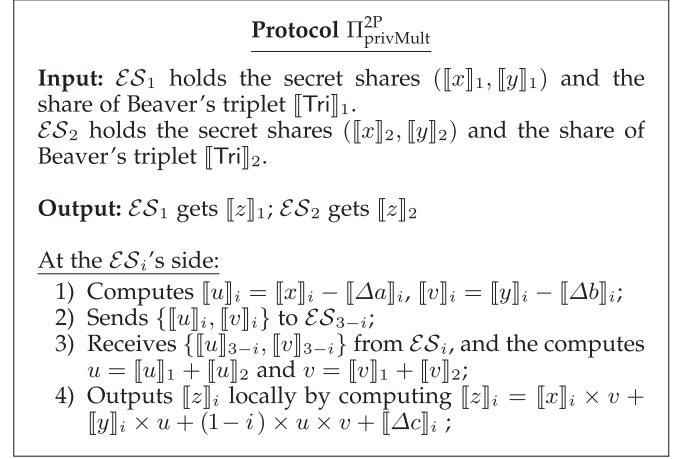


Fig. 2. Private multiplication protocol $\Pi_{\text{privMult}}^{2P}$ [22].

C. Private Truncation

As in the secure computation process, each decimal number $\tilde{v}$ is converted into an “integer” v , where the least significant l_D bits denote the fractional part, the protocol $\Pi_{\text{privMult}}^{2P}$ can also work on decimal numbers in the same manner. However, the length of fractional part doubles (i.e., $2 \times l_D$) and thus must *truncate* the least significant l_D bits to maintain l_D decimal bits for the product [26]. In this paper, we directly apply the secure truncation protocol (denoted as $\Pi_{\text{privTrunc}}^{2P}$) proposed by Mohassel and Zhang [36], that is, each participant truncates its share by right shifting l_D bits locally after every multiplication operation. The private truncation protocol can be executed as follows: given a secret share $[v']_i$, if $[v']_i$ denotes a positive number (i.e., $[v']_i < 2^{lp}$), $\mathcal{ES}_i$ computes $[v]_i := \lfloor \frac{[v']_i}{2^{l_D}} \rfloor$ locally; otherwise, it computes $[v]_i := \lfloor 2^l - \frac{2^l - [v']_i}{2^{l_D}} \rfloor$, where $\lfloor \cdot \rfloor$ denotes the round-down operation. We can find that the division operation for truncation can be locally performed with little computation overhead at each edge server's side.

Although this method would encounter a serious error when the most significant bit (MSB) of the secret v is different from that of both $[v]_1$ and $[v]_2$. Fortunately, Mohassel and Zhang [36] have proved that there is a negligible probability (i.e., 2^{lp-l}) of the error if $|v| < 2^{lp} \ll 2^l$ holds. The other inherent defect of this method is that it would removes a carry bit from the first l_D bit that would propagated into the $(l_D + 1)$ -th bit. However, Mohassel and Zhang also proved that the probability of this error is 2^{lD+1-l} , and actually the error is acceptable because it only causes a offset by 2^{-lD} .

D. Private Comparison

Given two secret values $u, v \in \mathbb{Z}_{2^l}$, we want to compute the comparison function as $f(u, v) = (u > v)$, where $f = 1$ if $u > v$ and $f = 0$ if $u \leq v$.

As the comparison function is vital for the computations in activation layers and maximum pooling layers of CNN model, the private comparison protocol will be one of the most significant building blocks to our proposed system. There are three types of solutions: 1) The first one is to use the garbled circuit proposed by

Yao [23] since it can evaluate any function represented as a boolean circuit. However, converting each arithmetic secret share into that in the form of Yao's garbled circuit will cost high bandwidth (i.e., $2l\kappa$ for the semi-honest adversary), where κ is the computational security parameter. 2) The second solution is based on the secure bit extraction protocol, such as that proposed by Huang *et al.* [22], while the protocol requires $(2l - 1)$ secure multiplications on the binary field, causing large burden to the latency-sensitive framework. 3) The third solution is to securely determine the sign of an intermediate result $z = r \times (u - v)$, where r is a random number to blind the value of $(u - v)$. However, it is easy to leak the information of $(u - v)$ on the condition that u is just equal to v , since the intermediate result z will be recovered by at least one participant.

To relieve the above problems, we design a new private comparison protocol denoted as $\Pi_{\text{privComp}}^{2P}$ shown in Fig. 3 to ensure the correctness of computation, there are some limitations to the inputs of the proposed protocol. Specifically, the absolute value of the input u and v can not exceed $2^{l/4}$ and the random integers generated in the procedure should also satisfy the requirement. This is because that we must ensure the absolute value of the intermediate result z_i computed at $\mathcal{ES}_i$'s side will not exceed $2^l - 1$. If no matter $f = 0$ or $f = 1$ when $u = v$, it does not affect the subsequent calculations, we can omit the two *alternative steps* (labeled in the Fig. 3) and let the output be $[f]_1 := [\alpha_1]_1 = e_1 \oplus k_1$, $[f]_2 := [\alpha_1]_2 = 1 \oplus k_2$ to reduce the rounds and bandwidth (here we define it as a simplified comparison protocol). We can observe that the simplified private comparison protocol only requires 3 rounds and at most $12l$ bits, which achieves a better tradeoff between the communication rounds and bandwidth consumption than some recently proposed protocols (more details can be referred in Section VII-A). Moreover, nothing can be leaked to the edge servers in the semi-honest adversary model.

For the first alternative step (i.e. the third round) that outputs $\{[\bar{z}_2]_1, [\bar{z}_2]_2\}$ and the second alternative step (i.e. the fifth round) that outputs $\{[f]_1, [f]_2\}$ is used to avoid the paradoxical result when $u = v$. In the other words, without the two rounds, the output f is completely a random value determined by $k_1 \oplus k_2$ when $u = v$. Trivially, the complete private comparison protocol $\Pi_{\text{privComp}}^{2P}$ can be used to securely compute the DReLU function in privacy-preserving training. Additionally, the second round and the third round for private multiplications can be performed simultaneously, and the exchanged messages for the two multiplications can be transmitted in one round to reduce network delay.

V. DESIGN OF PCNN_{ECE}

In this section, we present the detailed design of PCNN_{ECE}. The basic framework of PCNN_{ECE} are depicted in Fig. 4, and the detailed algorithms are described below.

A. Initialization and Secret Sharing

As the model parameters $\{\mathbf{W}, \mathbf{b}\}$ (including convolution kernels) are the service provider $\mathcal{SP}$'s private property which cannot be leaked to others. When an user $\mathcal{MU}$ wants to access

Protocol $\Pi_{\text{privComp}}^{2P}$

Input: $\mathcal{ES}_1$ holds the secret shares $([u]_1, [v]_1)$. $\mathcal{ES}_2$ holds the secret shares $([u]_2, [v]_2)$. Note that $|u| < 2^{l/4}$ and $|v| < 2^{l/4}$. $\mathcal{ES}_1$ and $\mathcal{ES}_2$ also have sufficient Beaver's triplet shares $\{(\Delta a_1, \Delta b_1, \Delta c_1), (\Delta a_2, \Delta b_2, \Delta c_2)\}$ in sequence, but we omit the description of input Beaver's triplets that will be used in Π_{privComp} for simplicity.

Output: $\mathcal{ES}_1$ gets $[f]_1$; $\mathcal{ES}_2$ gets $[f]_2$

The first round for $\mathcal{ES}_i$:

- 1) Generates two random negative integers $r_i, r'_i \in \mathbb{Z}_{2^{l/4}}$ and a random bit $k_i \in \mathbb{Z}_2$, where $r_i > r'_i$;
- 2) Computes $[u - v]_i = [u]_i - [v]_i$, $[v - u]_i = [v]_i - [u]_i$ and $s_i = (-1)^{k_i} r_i$;
- 3) Generates the additive secret shares $\{[s_i]_1, [s_i]_2\}$ and $\{[r'_i]_1, [r'_i]_2\}$ of s_i and r'_i respectively;
- 4) Sends $\{[s_i]_{3-i}, [r'_i]_{3-i}\}$ to $\mathcal{ES}_{3-i}$;

The second-fourth rounds for $\mathcal{ES}_i$:

- 1) Receives the shares $\{[s_{3-i}]_i, [r'_{3-i}]_i\}$ from $\mathcal{ES}_{3-i}$;
- 2) Computes $[s'_i]_i = [r_1]_i + [r_2]_i$ locally;
- 3) Runs the sub-protocol $\Pi_{\text{privMult}}^{2P}$ with $\mathcal{ES}_{3-i}$, where the inputs are $\{[s_1]_1, [s_2]_1\}, \{[s_1]_2, [s_2]_2\}$, and the results are $\{[t]_1, [t]_2\}$ for $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively, (i.e., $t = s_1 \times s_2$);
- 4) Runs the sub-protocol $\Pi_{\text{privMult}}^{2P}$ with $\mathcal{ES}_{3-i}$ again, where the inputs are $\{[t]_1, [u - v]_1\}, \{[t]_2, [u - v]_2\}$, and the results are $\{[\bar{z}_1]_1, [\bar{z}_1]_2\}$ for $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively;
- 5) Runs the sub-protocol $\Pi_{\text{privMult}}^{2P}$ with $\mathcal{ES}_{3-i}$ again, where the inputs are $\{[t]_1, [v - u]_1\}, \{[t]_2, [v - u]_2\}$, and the results are $\{[\bar{z}_2]_1, [\bar{z}_2]_2\}$ for $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively; (*This is an alternative step in some cases*);
- 6) Computes the share $[z_{3-i}]_i = [\bar{z}_{3-i}]_i - [s'_i]_i$ and sends it to $\mathcal{ES}_{3-i}$.

The fifth round for $\mathcal{ES}_i$:

- 1) Receives the shares $[z_{3-i}]_i$ from $\mathcal{ES}_{3-i}$;
- 2) Recovers z_i by computing $z_i = [z_i]_1 + [z_i]_2$;
- 3) Lets $e_i = 1$ if z_i denotes a negative number, otherwise $e_i = 0$;
- 4) Computes $[\alpha_i]_i = e_i \oplus k_i$, $[\alpha_{3-i}]_i = 1 \oplus k_i$;
- 5) Runs the sub-protocol $\Pi_{\text{privMult}}^{2P}$ with $\mathcal{ES}_{3-i}$ over the binary field $\mathbb{Z}_2$, where the inputs are $\{[\alpha_1]_1, e_1\}, \{[\alpha_1]_2, e_2\}$, and the results are $\{[f]_1, [f]_2\}$ for $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively; (*This is an alternative step sometimes*)

Fig. 3. Private comparison protocol $\Pi_{\text{privComp}}^{2P}$.

the service, it should first send its request message to $\mathcal{SP}$ for identity authentication.

If $\mathcal{SP}$ accepts the request message, it first locates two semi-honest and non-colluded edge servers (i.e., $\mathcal{ES}_1, \mathcal{ES}_2$) region-oriented toward the $\mathcal{MU}$. Next, $\mathcal{SP}$ secretly shares its model parameters as $\{\mathbf{W}, \mathbf{b}\}$ to $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively, where $\mathbf{W} = [\mathbf{W}]_1 + [\mathbf{W}]_2$ and $\mathbf{b} = [\mathbf{b}]_1 + [\mathbf{b}]_2$. Moreover, it generates enough number of Beaver's triplets for private multiplication operations, and sends them to the two edge servers respectively. We assume that the two edge servers will use the these triplets in sequence.

Given the two servers $\mathcal{ES}_1$ and $\mathcal{ES}_2$, the mobile user $\mathcal{MU}$ first performs a mutual authentication with the two edge

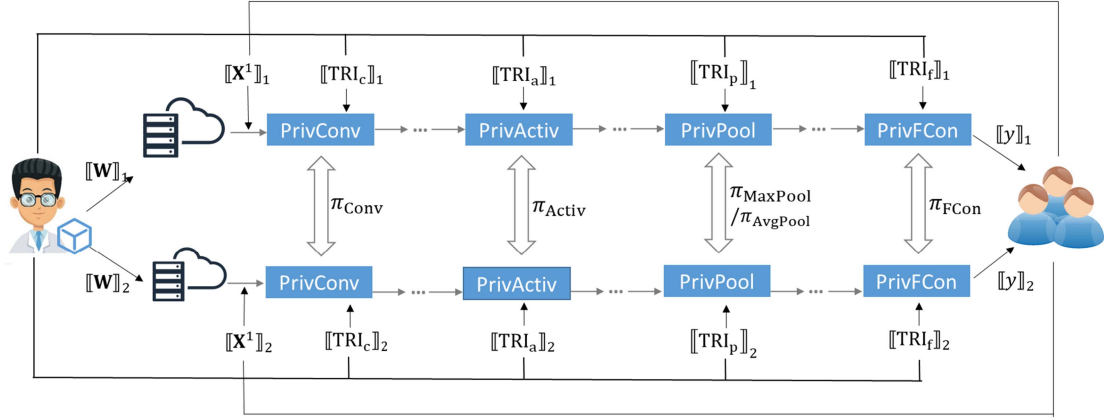


Fig. 4. Framework of the proposed scheme.

servers as well. Then, it splits its input feature vector $\mathbf{X}$ into two random parts $[\mathbf{X}]_1, [\mathbf{X}]_2$, where each element in $[\mathbf{X}]_i$ is chosen from the ring $\mathbb{Z}_{2^l}$ and satisfies that $\mathbf{X} = [\mathbf{X}]_1 + [\mathbf{X}]_2$. From the view of the two edge servers, the received model parameters and feature vectors are random numbers that leaks no information to the edge servers. And the Beaver's triplets are also generated randomly and independently from the other secret shares. Therefore, the initialization is privacy-preserving against the edge servers.

B. Private Convolution Computation

As described above, the convolution operation essentially performs matrix multiplications consisting of a sequence of inner products with multiple multiplications and additions. Intuitively, a private inner product consists of multiple private multiplications and additions, where the private multiplication uses the interactive sub-protocol $\Pi_{\text{privMult}}^{2P}$ and the private addition can be run locally. To reduce the rounds, we can run the multiple private multiplications in only one round as the idea proposed in [26].

In this phase, we assume that each edge server has converted all decimal values into the executable integers and represented all regions of a shared input volume $[\mathbf{X}^j]$ into a set of vectors as $\{[\mathbf{x}_0^j], \dots, [\mathbf{x}_{m-1}^j]\}$, where the size of $[\mathbf{x}_i^j]$ is the same with size of the corresponding kernel $[\mathbf{w}] \in \mathbb{Z}_{2^k}^{n \times n}$ in the current layer, such as $n \times n$. Assume there are d input volumes and enough number of shared triplets $[\text{TRI}] = \{([\Delta\mathbf{a}], [\Delta\mathbf{b}], [\Delta\mathbf{c}])\}$ have been shared to edge servers, where $([\Delta\mathbf{a}]_1 + [\Delta\mathbf{a}]_2) \times ([\Delta\mathbf{b}]_1 + [\Delta\mathbf{b}]_2) = ([\Delta\mathbf{c}]_1 + [\Delta\mathbf{c}]_2)$ (we omit the sub-scripts and sup-scripts of these vectors here). Then the private convolution computation is performed as Algorithm 1 shows.

From the Algorithm 1, we can see that it only needs one round to execute a private convolution computation. And each "for" statements of this algorithm can be executed in parallel to further speed up calculations in the network. Moreover, as the algorithm π_{Conv} usually works on decimal values, each edge server only needs to truncate the result $[\hat{z}_k^j]_i$ for each inner product after adding up all relative products. This can help to reduce the probabilistic errors caused by the inherent defects of the truncation protocol.

Algorithm 1 Private Convolution Computation π_{Conv}

Input: Each edge server $\mathcal{ES}_i$ inputs the volume shares $\{[\mathbf{X}^j]\}_{j=0}^{d-1}$, the kernel shares $\{[\mathbf{w}^j]\}_{j=0}^{d-1}$ and the triplet shares $[\text{TRI}]_i$.

Output: $\mathcal{ES}_i$ gets its own result share $\{[\mathbf{Z}^j]\}_{j=0}^{d-1}$.

- 1: **for** each volume $j \in \{0, \dots, d-1\}$ **do**
- 2: **for** each vector $k \in \{0, \dots, m-1\}$ **do**
- 3: $\mathcal{ES}_i$ computes the vector $[\mathbf{u}_k^j]_i = [\mathbf{x}_k^j]_i - [\Delta\mathbf{a}_k^j]_i$ and the vector $[\mathbf{v}_k^j]_i = [\mathbf{w}_k^j]_i - [\Delta\mathbf{b}_k^j]_i$;
- 4: **end for**
- 5: **end for**
- 6: $\mathcal{ES}_i$ sends a sequence of $\{([\mathbf{u}_k^j]_i, [\mathbf{v}_k^j]_i)\}_{j=0, k=0}^{d-1, m-1}$ to the other edge server $\mathcal{ES}_{3-i}$ simultaneously. In return, it receives a sequence of $\{([\mathbf{u}_k^j]_{3-i}, [\mathbf{v}_k^j]_{3-i})\}_{j=0, k=0}^{d-1, m-1}$;
- 7: **for** each volume $j \in \{0, \dots, d-1\}$ **do**
- 8: **for** each vector $k \in \{0, \dots, m-1\}$ **do**
- 9: $\mathcal{ES}_i$ computes $\mathbf{u}_k^j = [\mathbf{u}_k^j]_1 + [\mathbf{u}_k^j]_2$, $\mathbf{v}_k^j = [\mathbf{v}_k^j]_1 + [\mathbf{v}_k^j]_2$ and $[\hat{z}_k^j]_i = [\mathbf{x}_k^j]_i \cdot \mathbf{u}_k^j + [\mathbf{w}_k^j]_i \cdot \mathbf{v}_k^j + [\Delta\mathbf{c}_k^j]_i \cdot \mathbf{1}_k + (1-i)\mathbf{u}_k^j \cdot \mathbf{v}_k^j$;
- 10: $\mathcal{ES}_i$ executes the truncation operation for each $[\hat{z}_k^j]_i$ and gets $[\hat{z}_k^j]_i$ following the sub-protocol $\Pi_{\text{privTrunc}}^{2P}$;
- 11: **end for**
- 12: **end for**
- 13: $\mathcal{ES}_i$ re-arranges the vector $\{[\hat{z}_k^j]_i, \dots, [\hat{z}_{m-1}^j]_i\}$ (for each j in range d) be a new matrix as the next layer's input volume $[\mathbf{Z}^j]_i$;
- 14: **return** the result share $\{[\mathbf{Z}^j]_i\}_{j=0}^{d-1}$.

C. Private Activation Computation

Private ReLU Computation. The activate computation is a little more complex than other computations in the secure two-party setting, because it is usually a kind of non-linear computation. The piece-wise function ReLU is considered as a default recommendation in CNN. As the function ReLU can be mathematically defined as $y = \sigma(x) = \max\{x, 0\} = (x > 0 ? 1 : 0) \times x$ that combines a comparison and multiplication operations, thus to realize the private ReLU algorithm π_{Activ} , we only need to apply the private comparison protocol $\Pi_{\text{privComp}}^{2P}$ and the private multiplication protocol $\Pi_{\text{privMult}}^{2P}$.

Without loss of generality, we first introduce the private maximization operation between any two numbers x and y . Assume that each edge server has owned plenty of multiple triplets $[\text{TRI}]_i$, the private maximization computation works as the Algorithm 2 shows. Then, the private ReLU activation computation can be executed as the Algorithm 3 shows.

Algorithm 2 Private Maximum Computation π_{Max}

Input: Each edge server inputs its share $\llbracket x \rrbracket_i$, $\llbracket y \rrbracket_i$ and the triplet shares $\llbracket \text{TRI} \rrbracket_i$.

Output: Each edge server gets its result share $\llbracket z \rrbracket_i$.

- 1: The two edge servers $\mathcal{ES}_1$ and $\mathcal{ES}_2$ jointly runs the sub-protocol $\Pi_{\text{privComp}}^{2P}$, where the inputs are $\{\llbracket x \rrbracket_1, \llbracket y \rrbracket_1\}$, $\{\llbracket x \rrbracket_2, \llbracket y \rrbracket_2\}$ and the outputs are $\llbracket f \rrbracket_1$ and $\llbracket f \rrbracket_2$ respectively, and let $\llbracket u \rrbracket_1 := \llbracket f \rrbracket_1$, $\llbracket u \rrbracket_2 := \llbracket f \rrbracket_2$ respectively;
- 2: $\mathcal{ES}_1$ and $\mathcal{ES}_2$ jointly run the sub-protocol $\Pi_{\text{privMult}}^{2P}$, where the inputs are $\{(-1)^{\llbracket u \rrbracket_1} \llbracket x - y \rrbracket_1, (-1)^{\llbracket u \rrbracket_1}\}$, $\{(-1)^{\llbracket u \rrbracket_2} \llbracket x - y \rrbracket_2, (-1)^{\llbracket u \rrbracket_2}\}$ and the outputs are $\llbracket v \rrbracket_1, \llbracket v \rrbracket_2$ respectively;
- 3: $\mathcal{ES}_i$ locally computes $\llbracket z \rrbracket_i = \llbracket x \rrbracket_i - \frac{1}{2} \llbracket v \rrbracket_i$.

Considering the condition that if x is equal to zero, the output y will always be zero no matter what the result of the comparison protocol is. Thus, we can apply a *simplified private comparison protocol* that omits all the fifth steps (i.e., the alternative steps) described in $\Pi_{\text{privComp}}^{2P}$, then the outputs will be $\llbracket f \rrbracket_1 := \llbracket \alpha_1 \rrbracket_1 = e_1 \oplus k_1$, $\llbracket f \rrbracket_2 := \llbracket \alpha_1 \rrbracket_2 = 1 \oplus k_2$. In this way, we can save two rounds and some bandwidth when securely executing the function ReLU. That is, the activation computation of every number only needs 3 private multiplications and 4 rounds in all. As the inputs of each layer in CNN are usually vectors, so that multiple instances of the ReLU computations can be processed simultaneously, especially for the usage of private multiplications.

Private Leaky-ReLU Computation. As there are still some problems with ReLU activation function, such as exploding gradient and “dying ReLU,” Leaky-ReLU function is proposed. Instead of returning zero when the input value is negative, the Leaky-ReLU function returns a very small value which is α times of the negative value. It can be mathematically defined as $y = \sigma(x) = a^{1-(x>0 \text{ ? } 1:0)} \times x$, where a is a small decimal value less than 1. Since the value a is a constant, it can be known to everyone in a CNN training/prediction system. Therefore, the private Leaky-ReLU activation computation can be executed as follows: The two edge servers first jointly run the algorithm π_{Max} and gets the intermediate result $\llbracket z \rrbracket_1$ and $\llbracket z \rrbracket_2$ respectively, then each edge server $\mathcal{ES}_i$ locally computes its final output as

$$\llbracket y \rrbracket_i = a \llbracket x \rrbracket_i - (a - 1) \llbracket z \rrbracket_i \quad (4)$$

In practical evaluation, the equation (4) would be converted into $\llbracket y \rrbracket_i = \llbracket a \times 2^{l_D} \rrbracket_i - (\llbracket a \times 2^{l_D} \rrbracket_i - 2^{l_D}) \llbracket z \rrbracket_i$, so that each edge server has to run the private truncate operation on the output $\llbracket y \rrbracket_i$ and get the final activation result $\llbracket y \rrbracket_i$.

Correctness. Now, we first prove the private ReLU shown in the Algorithm 3 is correct. According to the property of the sub-protocol $\Pi_{\text{privMult}}^{2P}$, the outputs of the third step in Algorithm 3 satisfies that

$$\begin{aligned} v &= \llbracket v \rrbracket_1 + \llbracket v \rrbracket_2 \\ &= ((-1)^{\llbracket u \rrbracket_1} \llbracket x \rrbracket_1 + (-1)^{\llbracket u \rrbracket_2} \llbracket x \rrbracket_2) \times ((-1)^{\llbracket u \rrbracket_1} + (-1)^{\llbracket u \rrbracket_2}) \\ &= x + (-1)^u x \end{aligned} \quad (5)$$

Algorithm 3 Private Activation Computation With ReLU
Function π_{Acity}

Input: Each edge server inputs its share $\llbracket x \rrbracket_i$ and the triplet shares $\llbracket \text{TRI} \rrbracket_i$.

Output: Each edge server gets its result share $\llbracket y \rrbracket_i$.

- 1: The two edge servers $\mathcal{ES}_1$ and $\mathcal{ES}_2$ jointly generate a pair of random values $(\llbracket r \rrbracket_1, \llbracket r \rrbracket_2)$ satisfying that $r = 0$.
- 2: The two edge servers $\mathcal{ES}_1$ and $\mathcal{ES}_2$ jointly runs the private maximization algorithm π_{Max} as the Algorithm 2 defined, where the inputs are $\{\llbracket x \rrbracket_1, \llbracket r \rrbracket_1\}$, $\{\llbracket x \rrbracket_2, \llbracket r \rrbracket_2\}$ and the outputs are $\llbracket z \rrbracket_1$ and $\llbracket z \rrbracket_2$ respectively;
- 3: $\mathcal{ES}_i$ locally learns its output $\llbracket y \rrbracket_i := \llbracket z \rrbracket_i$.

Since $\llbracket u \rrbracket_1 := \llbracket f \rrbracket_1$, $\llbracket u \rrbracket_2 := \llbracket f \rrbracket_2$, we can know that if $x < 0$, then $f = \llbracket f \rrbracket_1 \oplus \llbracket f \rrbracket_2 = 0$. Thus, the value $u = \llbracket u \rrbracket_1 + \llbracket u \rrbracket_2$ would be zero or two, which makes the value v to be $2x$; otherwise, $f = 1$ and $u = 1$, which makes the value v to be zero. Then, based on the equation (5), the value y can be recovered as

$$\begin{aligned} y &= \llbracket y \rrbracket_1 + \llbracket y \rrbracket_2 = \llbracket x \rrbracket_1 - \frac{1}{2} \llbracket v \rrbracket_1 + \llbracket x \rrbracket_2 - \frac{1}{2} \llbracket v \rrbracket_2 \\ &= x - \frac{1}{2} v = x - \frac{1}{2} [x + (-1)^u x] \end{aligned} \quad (6)$$

If x is not a positive value, we have $y = x - \frac{1}{2}(2x) = 0$; otherwise, we have $y = x - \frac{1}{2} \times 0 = x$. Thus, the correctness of the private ReLU activation function is proved.

As for the private Leaky-ReLU, each edge server outputs $\llbracket y \rrbracket_i = a \llbracket x \rrbracket_i - (a - 1) \llbracket z \rrbracket_i$. And according to the equation (6), we can get $z = \frac{1}{2} x - (-1)^u x$. Thus, when the Leaky-ReLU function is applied in activation layers, the output satisfies

$$\begin{aligned} y &= \llbracket y \rrbracket_1 + \llbracket y \rrbracket_2 \\ &= a \llbracket x \rrbracket_1 - (a - 1) \llbracket z \rrbracket_1 + a \llbracket x \rrbracket_2 - (a - 1) \llbracket z \rrbracket_2 \\ &= ax + (a - 1)z \\ &= ax - (a - 1) \left[\frac{1}{2} x - (-1)^u x \right] \end{aligned} \quad (7)$$

which proves that $y = x$ if $x > 0$, and otherwise $y = ax$.

D. Private Pooling Computation

As described above, the pooling computation performs a downsampling operation on the spatial dimensions, and there are two popular pooling methods: one is the average pooling, the other is the maximum pooling. Assume that the size of each rectangular neighborhood to be downsampled is $n \times n$, and the values in the neighborhood can be represented as a tuple $\{x_0, x_1, \dots, x_{n^2-1}\}$.

Private AvgPool Computation. As for the average pooling (i.e., AvgPool), it is trial to compute the average value of the shared tuple $\{\llbracket x_0 \rrbracket, \llbracket x_1 \rrbracket, \dots, \llbracket x_{n^2-1} \rrbracket\}$ at each edge server's side. Hence, if we use the AvgPool in our network, each edge server $\mathcal{ES}_i$ can compute their own output value $\llbracket y \rrbracket_i = (\llbracket x_0 \rrbracket + \llbracket x_1 \rrbracket + \dots + \llbracket x_{n^2-1} \rrbracket) / n^2$ locally. With this method, the private CNN prediction network will become very

Algorithm 4 Private Maximum Pooling Computation π_{MaxPool}

Input: Each edge server inputs its shares $[\mathbf{x}]_i = \{[x_j]_i\}_{j=0}^{n^2-1}$ and the triplet shares $[\text{TRI}]_i$.

Output: Each edge server gets its result share $[y]_i$.

- 1: Each edge server $\mathcal{ES}_i$ first lets $[y]_i := [x_0]_i$, then it jointly runs the following steps with $\mathcal{ES}_{3-i}$;
- 2: **for** each $j \in \{1, 2, \dots, n^2 - 1\}$, **do**
- 3: $\mathcal{ES}_1$ and $\mathcal{ES}_2$ jointly run the algorithm π_{Max} , where the inputs are $\{[y]_1, [x_j]_1\}$, $\{[y]_2, [x_j]_2\}$ and the intermediate outputs are $[z]_1$ and $[z]_2$ respectively;
- 4: $\mathcal{ES}_i$ lets $[y]_i := [z]_i$;
- 5: **end for**
- 6: $\mathcal{ES}_i$ locally learns its final output $[y]_i$.

efficient, since the private average pooling algorithm π_{AvgPool} does not cost any round or bandwidth.

Private MaxPool Computation. Some prediction network would use the maximum pooling method MaxPool, which is much more complicated in secure two-party setting. The maximum pooling computation for each rectangular neighborhood involves multiple special comparison operations: finding the maximum between a pair of numbers. Specifically, for any two numbers (x, y) , we present the private maximum computation procedure in Algorithm 2. Then, after performing $n^2 - 1$ such private comparison operations, the two edge servers can get their final result $[z]_1, [z]_2$ respectively, satisfying that $[z]_1 + [z]_2 = \max\{x_0, x_1, \dots, x_{n^2-1}\}$. For each rectangular neighborhood, the private maximum pooling computation can be executed as the Algorithm 4 shows.

To reduce the running rounds, some instances of the algorithm π_{Max} can be executed in parallel. For example, given a tuple $\mathbf{x}$ of size 6, the operations $\pi_{\text{Max}}(x_0, x_1)$, $\pi_{\text{Max}}(x_2, x_3)$ and $\pi_{\text{Max}}(x_4, x_5)$ can be executed simultaneously. Therefore, if the size of a rectangular neighborhood is n^2 , the number of rounds to perform the maximum pooling will be $4\lceil\log(n^2)\rceil$.

E. Private Full-Connection Computation

The full-connection computation in essence performs several inner products between the input data and the weight parameters at the last layer. Hence, the procedures of private full-connected computation π_{FCOn} is similar to that of the private convolutional computation shown in Algorithm 3. To avoid repetition, we do not describe the details here. Specifically, each edge server's inputs are denoted as $[\mathbf{X}]_i$, $[\mathbf{w}]_i$ and $[\text{TRI}]_i$, and the output is denoted as $[\mathbf{fr}]_i$. $\mathcal{ES}_1$ and $\mathcal{ES}_2$ independently send their outputs $[\mathbf{fr}]_1$ and $[\mathbf{fr}]_2$ to the mobile user in a secure manner, such as encrypting them using an authenticated session key generated in the initialization phase.

F. Predication Result Recovery

Upon receiving the encoded messages from the two edge servers $\mathcal{ES}_1$ and $\mathcal{ES}_2$, the mobile user first decodes the messages and gets the results $[\mathbf{fr}]_1$ and $[\mathbf{fr}]_2$, then it only needs to perform an element-wise addition operation such that $\mathbf{fr} = [\mathbf{fr}]_1 + [\mathbf{fr}]_2$. With the recovered vector $\mathbf{fr}$, the user can surely

get the final prediction result according to some certain regulation. We can observe that in the whole prediction procedures, except the mutual authentication process, the mobile user only needs to do some simple addition and subtraction operations in the initialization phase and the recovery phase, which is computation-friendly to the mobile user. And the data privacy is ensured by the secure two-party computation.

VI. SECURITY ANALYSIS

In this section, we prove that the proposed PCNN_{CEC} achieves perfect privacy in the presence of semi-honest adversaries. From the specification of our proposed PCNN_{CEC}, both the mobile user's data and the service provider's model parameters are randomly shared and independently distributed to two non-colluded edge servers, the privacy of the model and user data is preserved. During the prediction procedures, both the edge servers always process them and their intermediate values over secret shares with the assistance of Beaver's triplets. Although the triplets are generated by the service provider, it is still unable to violate the user's privacy, because the service provider does not involve the following prediction procedure and the edge servers can only know their own triplets.

Below we provide simulation-based security proofs in the universal composability framework [42], [43]. Specifically, we follow the security definition and lemmas presented in prior works such as [21], [22], [44]

Security Definition. Let $f = (f_1, f_2)$ be a probabilistic polynomial-time functionality and let Π be a two-party protocol for computing f . And let $\text{view}_i^\Pi(x_1, x_2, l)$ denote the view of $\mathcal{ES}_i$ in the real world, where x_1, x_2 denote the inputs of $\mathcal{ES}_1$ and $\mathcal{ES}_2$ respectively. We say that the protocol f is secure in the presence of semi-honest adversaries if there exists probabilistic polynomial-time simulators $\mathcal{S}_1$ and $\mathcal{S}_2$ such that

$$\mathcal{S}_1(1^l, x_1, f_1(x_1, x_2)) \stackrel{c}{=} \text{view}_1^\Pi(x_1, x_2, l)$$

$$\mathcal{S}_2(1^l, x_2, f_2(x_1, x_2)) \stackrel{c}{=} \text{view}_2^\Pi(x_1, x_2, l)$$

where $\mathcal{S}_i(1^l, x_i, f_i(x_1, x_2))$ denotes the view of $\mathcal{A}$ generated by $\mathcal{S}_i$, and $\stackrel{c}{=}$ denotes computational indistinguishability.

Lemma 1: The whole system is simulatable if all the building blocks are simulatable.

Lemma 2: The element $x \cdot r$ (or $x \pm r$) is random and independent of x if the value r is a uniformly distributed random element chosen in $\mathbb{Z}_{2^l}$.

Lemma 3: $\Pi_{\text{privMult}}^{2P}, \Pi_{\text{privTrunc}}^{2P}$ are secure in the presence of semi-honest adversaries, and the protocols built on their linear combinations are secure as well.

Because the service provider and IoT devices only need to execute some data sharing and recovering operations, we mainly focus on the provable security of those secure two-party computations executed by $\mathcal{ES}_1$ and $\mathcal{ES}_2$. The above lemmas will be used to prove the following theorems that illustrate the security of the proposed $\Pi_{\text{privComp}}^{2P}$ and PCNN_{CEC}.

Theorem 1: The protocol $\Pi_{\text{privComp}}^{2P}$ is secure in the presence of semi-honest adversaries.

Proof: $\mathcal{ES}_1$ and $\mathcal{ES}_2$ collaboratively perform the comparison protocol with the help of some multiplication triplets. From the specification of $\Pi_{\text{privComp}}^{2P}$, the view for $\mathcal{ES}_i$ is $\text{view}_i^\Pi([u]_i, [v]_i, \{\Delta a_i, \Delta b_i, \Delta c_i\}, [s_i]_{3-i}, [r'_i]_{3-i}, k_i, r_i, r'_i, [t]_i, [\tilde{z}_1]_i, [\tilde{z}_2]_i, [\alpha]_i, e_i)$. We can observe that (k_1, r_i, r'_i) are random numbers generated by $\mathcal{ES}_i$ and independently from $[u]_i, [v]_i$; For $\{\Delta a_i, \Delta b_i, \Delta c_i\}$, they are the triplets of $\mathcal{S}_i$ which can be also considered as random numbers at the view of $\mathcal{ES}_{3-i}$; While $([s_i]_{3-i}, [r'_i]_{3-i})$ and $([t]_i, [\tilde{z}_1]_i, [\tilde{z}_2]_i, [\alpha]_i, e_i)$ are also the intermediate values, the former are linear combinations of (k_1, r_i, r'_i) , the later are obtained from the protocol $\Pi_{\text{privMult}}^{2P}$. Thus, we can construct the simulator $\mathcal{S}_i([u]_i, [v]_i)$ as:

- 1) Chooses several sets of $\{\Delta \hat{a}_i, \Delta \hat{b}_i, \Delta \hat{c}_i\}$, which are all random integers from $\mathbb{Z}_{2^l}$.
- 2) Chooses random numbers $\hat{r}_i, \hat{r}'_i$ from $\mathbb{Z}_{2^l}$ and a random bit $\hat{k}_i \in \{0, 1\}$;
- 3) Computes the intermediate value $[\hat{s}_i]_{3-i}, [\hat{r}'_i]_{3-i}$ with the chosen random numbers $(\hat{r}_i, \hat{r}'_i, \hat{k}_i)$ following the specifications of protocol $\Pi_{\text{privComp}}^{2P}$;
- 4) Runs the private multiplication protocol following the specification of $\Pi_{\text{privMult}}^{2P}$ with the numbers chosen in step 1), and then gets the intermediate values $[\hat{t}]_i, [\hat{z}_1]_i, [\hat{z}_2]_i, [\hat{\alpha}]_i, \hat{e}_i$.

Since the triplets are generated randomly by the service provider, so the distribution of $\{\Delta \hat{a}_i, \Delta \hat{b}_i, \Delta \hat{c}_i\}$ is indistinguishable from that in real execution. Because $\hat{r}_i$ (resp. $\hat{r}'_i, \hat{k}_i$) is also generated in the same manner with r_i (resp. r'_i, k_i), they are indistinguishable in distributions. Based on the *Lemma 2*, the distributions of intermediate value $[\hat{s}_i]_{3-i}$ (resp. $[\hat{r}'_i]_{3-i}$) and $[s_i]_{3-i}$ (resp. $[r'_i]_{3-i}$) are also indistinguishable. And based on the *Lemma 3*, the computations of $\Pi_{\text{privMult}}^{2P}$ are proved to be simulatable and secure in the presence of semi-honest adversaries, hence the number $[\hat{t}]_i$ (resp. $[\hat{z}_1]_i, [\hat{z}_2]_i, [\hat{\alpha}]_i, \hat{e}_i$) is indistinguishable from $[t]_i$ (resp. $[z_1]_i, [z_2]_i, [\alpha]_i, e_i$) in distribution as well. Therefore, based on the *Lemma 1*, the view of a corrupted $\mathcal{ES}_{3-i}$ in the real world is distinguishable from that in the simulated world created by $\mathcal{S}_i$. That is, $\text{view}_i^\Pi([u]_i, [v]_i, \{\Delta a_i, \Delta b_i, \Delta c_i\}, [s_i]_{3-i}, [r'_i]_{3-i}, k_i, r_i, r'_i, [t]_i, [\tilde{z}_1]_i, [\tilde{z}_2]_i, [\alpha]_i, e_i) \equiv \mathcal{S}_i([u]_i, [v]_i, \{\Delta \hat{a}_i, \Delta \hat{b}_i, \Delta \hat{c}_i\}, [\hat{s}_i]_{3-i}, [\hat{r}'_i]_{3-i}, \hat{k}_i, \hat{r}_i, \hat{r}'_i, [\hat{t}]_i, [\hat{z}_1]_i, [\hat{z}_2]_i, [\hat{\alpha}]_i, \hat{e}_i)$.

Theorem 2: The private convolution computation algorithm π_{Conv} is secure in the presence of semi-honest adversaries.

Proof: According to the Algorithm 1, the two edge servers perform the convolutional computation with the pre-computed triplets. As $\mathcal{ES}_i$'s private inputs are $\{\mathbf{X}^j\}_{j=0}^{d-1}$ and $[\text{TRI}]_i$, where $\mathbf{X}^j = \{\mathbf{x}_k^j\}_{k=0}^{m-1}$ and $[\text{TRI}]_i$ consists of sufficient triple vectors such as $\{\Delta \mathbf{a}_k^j, \Delta \mathbf{b}_k^j, \Delta \mathbf{c}_k^j\}$, the view of $\mathcal{ES}_i$ is $\text{view}_i^\Pi(\{\mathbf{x}_k^j\}, \{\mathbf{w}^j\}_i, [\text{TRI}]_i, \{\mathbf{u}_k^j\}_{3-i}, \{\mathbf{v}_k^j\}_{3-i})$ for $j \in [0, d-1], k \in [0, m-1]$. Then, for each (k, j) , we can construct a simulator $\mathcal{S}_i([\mathbf{x}_k^j]_i, [\mathbf{w}^j]_i)$ as follows:

- 1) Picks random integers $[\hat{\mathbf{u}}_k^j]_{3-i}, [\hat{\mathbf{v}}_k^j]_{3-i}$ from the ring $\mathbb{Z}_{2^l}$ and randomly generates some triplet sets $[\hat{\text{TRI}}]_i$;
- 2) Computes $\hat{\mathbf{u}}_k^j = [\mathbf{x}_k^j]_1 - [\Delta \hat{\mathbf{a}}_k^j]_1 + [\hat{\mathbf{u}}_k^j]_2$ and $\hat{\mathbf{v}}_k^j = [\mathbf{w}^j]_1 - [\Delta \hat{\mathbf{b}}_k^j]_1 + [\hat{\mathbf{v}}_k^j]_2$;

- 3) Computes $[\hat{z}_k^j]_i = [\mathbf{x}_k^j]_i \cdot \hat{\mathbf{u}}_k^j + [\mathbf{w}^j]_i \cdot \hat{\mathbf{v}}_k^j + [\Delta \hat{\mathbf{c}}_k^j]_i \cdot \mathbf{1}_k^j + (2-i)\hat{\mathbf{u}}_k^j \cdot \hat{\mathbf{v}}_k^j$;
- 4) Outputs $\mathcal{S}_i([\mathbf{x}_k^j]_i, [\mathbf{w}^j]_i, [\hat{\text{TRI}}]_i, \hat{\mathbf{u}}_k^j, \hat{\mathbf{v}}_k^j, [\hat{z}_k^j]_i)$.

According to the *Lemma 2*, $[\hat{\mathbf{u}}_k^j]_{3-i} = [\mathbf{x}_k^j]_{3-i} - [\Delta \hat{\mathbf{a}}_k^j]_{3-i}$ is uniformly random and independent of $[\mathbf{x}_k^j]_i$, since $[\Delta \hat{\mathbf{a}}_k^j]_i$ is random and independent from $[\mathbf{x}_k^j]_i$. Analogously, $[\hat{\mathbf{v}}_k^j]_i$ is uniformly random and independent of $[\mathbf{w}^j]_i$. And in the real execution, $\{\Delta \mathbf{a}_k^j, \Delta \mathbf{b}_k^j, \Delta \mathbf{c}_k^j\}$ are also random values that are independent of the private inputs $([\mathbf{x}_k^j]_i, [\mathbf{w}^j]_i)$ at the view of a corrupted $\mathcal{ES}_{3-i}$. Hence, the distributions of $[\hat{\mathbf{u}}_k^j]_i$ (resp. $[\hat{\mathbf{v}}_k^j]_i, [\hat{z}_k^j]_i$) and $[\mathbf{u}_k^j]_i$ (resp. $[\mathbf{v}_k^j]_i, [\mathbf{z}_k^j]_i$) are indistinguishable. That is, $\text{view}_i^\Pi(\{\mathbf{x}_k^j\}, \{\mathbf{w}^j\}_i, [\text{TRI}]_i, \{\mathbf{u}_k^j\}_{3-i}, \{\mathbf{v}_k^j\}_{3-i}) \stackrel{c}{=} \mathcal{S}_i(\{\mathbf{x}_k^j\}_i, \{\mathbf{w}^j\}_i, [\hat{\text{TRI}}]_i, \{\hat{\mathbf{u}}_k^j\}_{3-i}, \{\hat{\mathbf{v}}_k^j\}_{3-i})$.

In the other hand, a private convolution computation is identical to a sequence of private multiplications that are executed simultaneously. While the *Lemma 3* has claimed that the private multiplication protocol is secure in the presence of semi-honest adversaries, hence the private convolution computation algorithm is also secure in the presence of semi-honest adversaries.

Theorem 3: The private activation computation algorithm π_{Activ} is secure in the presence of semi-honest adversaries.

Proof: Given a secret input x for the a private activation computation, the view of $\mathcal{ES}_i$ is $\text{view}_i^\Pi([x]_i, [r]_i, [z]_i, [y]_i)$. We can construct a simulator $\mathcal{S}_i([x]_i)$ as below:

- 1) Computes $\hat{r}_i = 2^l - [r]_{3-i}$ upon receiving the share $[r]_{3-i}$ from a corrupted $\mathcal{ES}_{3-i}$;
- 2) Runs the private maximum computation π_{Max} according to the specification described in Algorithm 2, and gets the intermediate value $[\hat{z}]_i$;
- 3) Computes $[\hat{y}]_i = a[x]_i - (a-1)[z]_i$ if the activation function is leaky-ReLU, otherwise $[\hat{y}]_i = [\hat{z}]_i$ when the activation function is ReLU.

In the real execution, $[r]_1$ and $[r]_2$ such that $r = 0 \pmod{2^l}$ are jointly generated by the two edge servers, so we can consider the two shares are public to each edge server. Thus, the number $[\hat{r}]_i$ in the simulated world is indistinguishable from $[r]_i$ in the real execution. As the Algorithm π_{Max} composes of a private comparison protocol $\Pi_{\text{privComp}}^{2P}$ (which is proved to be secure in *Theorem 1*) and a private multiplication protocol $\Pi_{\text{privMult}}^{2P}$ (which is proved to be secure in *Lemma 3*), the maximum algorithm π_{Max} is secure in the presence of semi-honest adversaries according to the *Lemma 1*. Hence, the number $[\hat{z}]_i$ is simulatable and private at the view of a corrupted $\mathcal{ES}_{3-i}$, which leads the number $[\hat{y}]_i$ also being secure according to the *Lemma 2*. Therefore, based on the *Lemma 1*, we can have $\text{view}_i^\Pi([x]_i, [r]_i, [z]_i, [y]_i) \stackrel{c}{=} \mathcal{S}_i([x]_i, [\hat{r}]_i, [\hat{z}]_i, [\hat{y}]_i)$.

Theorem 4: The private pooling computation algorithm $\pi_{\text{AvgPool}} / \pi_{\text{MaxPool}}$ is secure in the presence of semi-honest adversaries.

Proof: For concisely, we here omit the trivial simulation proof for the private pooling computation. We directly analyze it security based on the above lemmas and theorems. For the private average pooling algorithm, it only has a local

computation at each edge server's side, so that it is surely to be secure in the presence of semi-honest adversaries. As for the private π_{MaxPool} , it consists of a sequence of private maximum algorithm π_{Max} , which is proved to be secure in the presence of semi-honest adversaries in *Theorem 3*. Therefore, π_{MaxPool} is also secure. ■

Theorem 5: The private fully connected computation algorithm π_{FCOn} is secure in the presence of semi-honest adversaries.

Proof: As a private fully connected computation is also a set of private multiplications being executed simultaneously, which is to the procedures of the convolutional computation, so we consider the private fully connected computation algorithm is secure in the presence of semi-honest adversaries. For brevity, we here omit the concrete simulation proofs as well. ■

As last, since each phase of the system is proved to be secure, we can say the whole system PCNN_{CEC} is secure according to the the universally composable model.

VII. PERFORMANCE EVALUATION

In this section, we perform some experiments to demonstrate the efficiency and accuracy of PCNN_{CEC} and its core underpinning protocol $\Pi_{\text{privComp}}^{2P}$.

Databases and Network Architectures. Following the prior works such as [21], [22], we also use two standard databases MNIST [45] and CIFAR-10 [46] to conduct our experiments. The MNIST database is used for handwritten digit classification consisting of 60,000 training images and 10,000 test images, and each greyscale image is 28×28 representing the digits 0-9. The CIFAR-10 database consists of 60000 32×32 color images in 10 classes, consisting of 6,000 training images and 1,000 test images per class. Regardless the realistic databases, a practical CNN model is also an important factor to demonstrate the system performance. We also use the three networks shown in Huang *et al.*'s scheme [22] (i.e., Network-I, Network-II, Network-III). The former two are trained for MNIST recognition task, and the last one is trained for CIFAR-10 classification task.

Experimental Setup. In our experiments, we assume that the user data and model parameters, multiplication triplets have been secretly shared to the two edge servers. Then, we use two laptops to simulate the edge servers, one of them is equipped with an Intel(R) Core(TM) i7-7600 CPU operating at 2.80GHz with 16GB RAM, the other is equipped with an Intel(R) Core(TM) i7-8565U CPU operating at 1.80GHz with 8GB RAM in Ubuntu. Our benchmarks are run in two networks settings, namely, the LAN network setting with two laptops in Wuhan, China, while the WAN networking setting with one laptop in Wuhan and the other laptop in Shanghai, China.

Huang *et al.* [22] have proved that when $l_D \geq \log 10$, the accuracy can be maintained as the same with that obtained in plaintext. Thus, the length of ring modulo l is set to be 32-bit, and the length of decimal part l_D is set to be 5-bit. We implement our system in C++ with Python binding. And some two-party computations are conducted based on

TABLE II
COMPARISON OF COMMUNICATION WITH PRIOR WORKS FOR PRIVATE
COMPARISON PROTOCOL $\Pi_{\text{privComp}}^{2P}$

Works	Comm. (bits)	Rounds
MiniONN [38]	$4\lambda l$	2
Huang et al. [22]	$8l - 4$	l
Li et al. [21]	—	—
CryptFlow2 [20]	$\lambda l + 14l$	$\log l$
This work	$16l$	5
This work (Simplified)	$12l$	3

The parameter l denotes the length of an integer in the ring $\mathbb{Z}_{2^l}$, while λ is the security parameter and typically 128.

the open source library EMP-Toolkit [47]. We repeated each experiment at 5 times and calculated the mean run-time as the result.

A. Performance of Private Comparison

In this work, we can observe that the linear computations such as additions and multiplications can be performed efficiently based on the arithmetic sharing [39], which are also widely used in other similar works. And the non-linear comparison computation, as the core of ReLU / leaky-ReLU activation computations and the maximum pooling computations, will become the main part to influence the efficiency of a private CNN network. Hence, we first analyze the communication and computation costs of the proposed private comparison protocol. Moreover, to further demonstrate the efficiency, we compare it with some recently proposed and related methods in similar works, including GC-based MiniONN [38], oblivious transfer based CryptFlow2 [20] and ASS-based Li *et al.* [21] and Huang *et al.* [22].

Table II shows the theoretic analysis results for different private comparison methods. MiniONN [38] uses a garbled circuit to perform the private comparison operation that only requires 2 rounds but has to cost approximately $4\lambda l$ bits and it requires additional bandwidth and rounds to execute the transformation operations between arithmetic sharing and garbled circuits. Hence, the whole comparison communication overhead of MiniONN may be $(7\lambda l + \frac{l^2+1}{2})$ bits at 4 rounds for ring $\mathbb{Z}_{2^l}$. CryptFlow2 [20] optimizes the technique of MiniONN to reduce the bandwidth at $(\lambda l + 14l)$ bits but increases the rounds at $\log l$. Huang *et al.* [22] uses the bit de-composition technique to implement the private comparison protocol that only needs to cost $(8l - 4)$ bits, but requires l rounds. Li *et al.* [21] do not directly apply a private comparison protocol in their scheme. In this paper, we proposed a new two-party secure comparison protocol that costs at most $16l$ bits and 5 rounds to run a quite accurate comparison. However, the simplified comparison protocol that removes the two



Fig. 5. Comparison of bandwidths for $\Pi_{privComp}^{2P}$.

alternative rounds will only costs approximately $12l$ bits within 3 rounds. Fig. 5 shows the communication costs of related works with different value of l . We can observe that the communication cost of our private comparison protocol is a little higher than Huang *et al.*'s proposal, but lower than other protocols.

To further demonstrate the efficiency of our private comparison protocol, we compare the runtime with those in the above related works. Assume that the multiplication triplets have been prepared before executing the comparison operation, we only evaluate the (online) runtime of these comparison protocols in LAN setting and WAN setting respectively. The experimental results are shown in Fig. 6, we can observe that if the private comparison protocol is only run once, the method of MiniONN is the most efficient, and our private comparison protocol is more efficient than that in Huang *et al.*'s scheme and CryptFlow2. Otherwise, in the LAN setting, if the parallel batch size is bigger than 10 but smaller than 10000, the proposed $\Pi_{privComp}^{2P}$ is more efficient than other works. In the WAN setting, if the parallel batch size is larger than 100, our proposed protocol $\Pi_{privComp}^{2P}$ may be the most effective protocol. In reality, it is easy to achieve such a batch size that larger than one. However, it is hard to let the batch size more than 10000, since few machines can handle such resource load. Therefore, our proposed private comparison protocol $\Pi_{privComp}^{2P}$ can achieve a much better efficiency than other methods in reality.

B. Performance of PCNN_{CEC}

To demonstrate the performance of PCNN_{CEC}, we also compare our proposal with three similar works including MiniONN [38], Huang *et al.*'s scheme [22], Li *et al.*'s scheme [21] and CryptFlow2 [20]. Before evaluate their performances, we list the security properties that these schemes can meet in Table III.

Comparison for ReLU. For a privacy-preserving neural network inference, the non-linear activation computation domain the whole runtime since it requires more rounds than other linear transformation computations. By referring to the literature [20], the GC-based privacy-preserving ReLU function has to cost $(8\lambda l - 4\lambda)$ bits for $\mathbb{Z}_{2^l}$, but only needs 2 rounds. Hence,

MiniONN and Li *et al.*'s scheme [21] have to costs $(8\lambda l - 4\lambda)$ bits within 2 rounds for a privacy-preserving ReLU computation, and they can perfectly guarantee the privacy of ReLU function's outputs. CryptFlow2 needs to cost about $(\lambda l + 18l)$ with $(\log l + 2)$ rounds. In Huang *et al.* scheme [22], the two edge servers exchange their outputs of private comparison protocol, and then gets the shares of ReLU function. Hence, Huang *et al.*'s scheme greatly reduces the network bandwidth consumption at about $(10l - 4)$ bits. However, Huang *et al.*'s scheme requires $(l + 1)$ rounds, and can only achieve low-level privacy, because both the two edge servers can recover the comparison results. As for our proposed π_{Activ} , it will cost approximately $13l$ bits with 4 rounds, and can perfectly protect the privacy of comparison results during ReLU computations.

Fig. 7 shows the comparisons of communication costs with different integer length l . We can observe that the larger l is, the more bandwidth will be consumed. As MiniONN is designed over the field $\mathbb{Z}_n$ where the integer length is $\eta = \lceil \log n \rceil$, we set $\eta = 10, 11, 12, 13$ corresponding to $l = 8, 16, 32, 64$. Then, we can observe that MiniONN and Li *et al.*'s scheme are similar and cost more communication overheads when compared with other schemes. Huang *et al.*'s scheme costs the least bandwidth, and our proposed PCNN_{CEC} is only a little higher than their scheme.

Next, we also launched some experiments to evaluate the runtime of these protocols as shown in Fig. 8. Similar to the private comparison protocol, if the protocol only needs to be run once, the runtime of our proposed $\pi_{\text{privActiv}}$ will be a little higher than that in MiniONN, but will be lower than that in Huang *et al.*'s scheme and CryptFlow2. However, when the protocol is run in parallel with the batch size at $10 \leq bs \leq 5000$, our proposed private ReLU protocol will be the most efficient one in the LAN setting. If the batch size is approximately greater than 300, the proposed private ReLU protocol will be the most efficient in the WAN setting. Considering the reality of the situation, the private ReLU computation of a private inference system will be usually run in parallel with a desirable batch size. Therefore, the proposed private ReLU has a promising potential in efficient and privacy-preserving CNN inference.

Comparison for Pool. For the average pooling method, CryptFlow2 has to cost about $(\lambda + 2l) \cdot (l + \delta)$ bits at $\log(l\delta) + 4$ rounds. While in the other related works and our proposed PCNN_{CEC}, the two edge servers only need to compute the mean value of their shares locally. For the max pooling method, Li *et al.* did not use it in their scheme, while the other works implement the privacy-preserving max pooling computation based on their proposed private comparison protocols. The theoretical comparison result of communications for π_{MaxPool} is presented in Table V. As a MaxPool computation can be considered as some comparison operations and ReLU operations running in parallel, we here not conduct concrete experiments for it. Theoretically, we can observe that the bandwidth consumption of our proposed π_{MaxPool} is higher than Huang *et al.*'s scheme, but much lower than other schemes, and the interactive latency of our work is quite less than Huang *et al.*'s protocol.

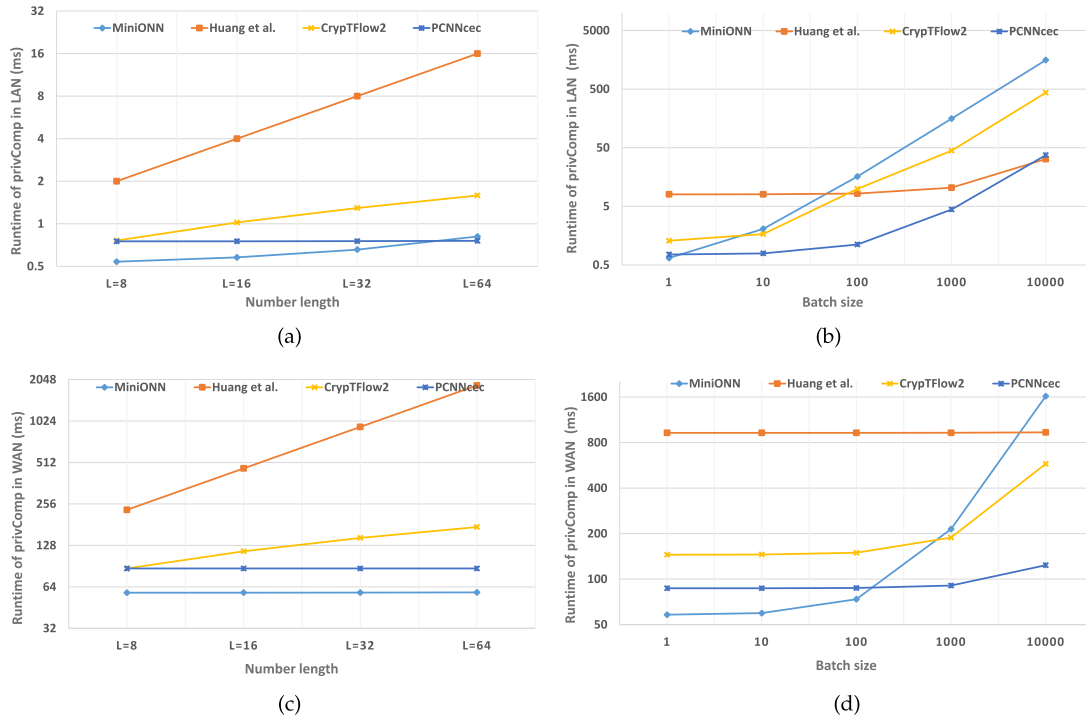


Fig. 6. Runtime of privacy-preserving comparison protocols: (a)Runtime for different bit-width l in LAN; (b)Runtime for different parallel batch size bs in the LAN setting with $l = 32$; (c)Runtime for different bit-width l in WAN; (d)Runtime for different parallel batch size bs in the WAN setting with $l = 32$.

TABLE III
COMPARISON OF SECURITY LEVELS FOR RELATED WORKS

Properties	MiniONN [38]	Huang et al. [22]	Li et al. [21]	CryptFlow2 [20]	PCNN _{CEC}
Data privacy	✓	✓	✓	✓	✓
Model privacy	×	×	✓	✓	✓

✓ denotes that the security property can be achieved; × denotes that the scheme does not meet the property.

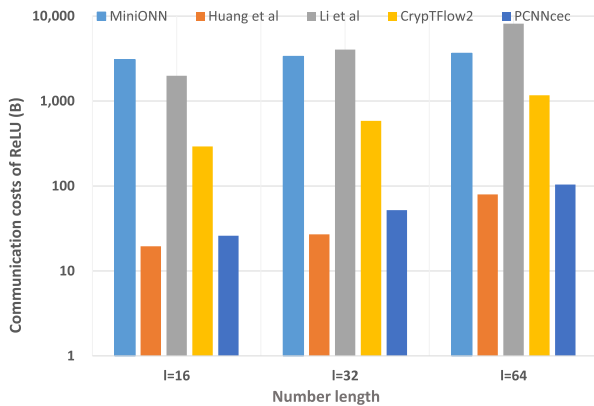


Fig. 7. Comparison of communication costs with different l .

Efficiency of PCNN_{CEC}. Tables VI and VII presents the online runtime of each stage in the LAN setting and WAN setting respectively. From the results, we can observe that the communication cost of each layer dominates the runtime in the LAN setting. And in the WAN setting, both the communication cost

and rounds of influence the efficiency. For network I on the MNIST dataset, it only needs 18 milliseconds and 188 milliseconds to complete a privacy-preserving prediction in LAN and WAN settings respectively. For network II, it respectively costs about 175 milliseconds and 1.808s in the LAN and WAN settings. Network III is more complex that it requires about 19.439 seconds and 63.492 seconds to in LAN and WAN settings respectively. To further demonstrate the efficiency of PCNN_{CEC}, we compare its runtime with other schemes when running in the network II and network III, because the two networks are more representative.

From Table III, we can see that Huang *et al.*'s scheme cannot achieve model privacy, thus we do not consider it in the efficiency comparisons when running with the realistic datasets. CryptFlow2 applies the oblivious transfer and homomorphic encryption to execute the linear matrix multiplication operations, which may incur more computation costs and communication overhead than those based on additively arithmetic secret sharing technology. Thus, we here mainly compare PCNN_{CEC} with MiniONN and Li *et al.*'s scheme. Note that in Network-II,

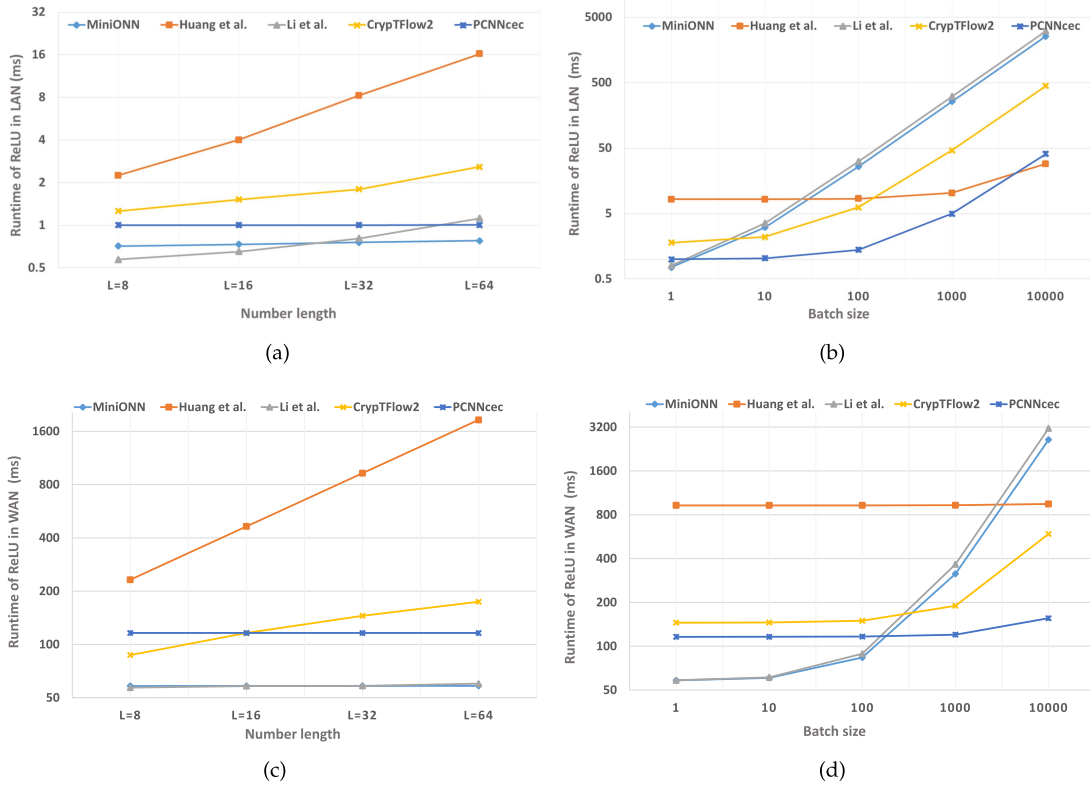


Fig. 8. Performance of private ReLU: (a)Runtime for different bit-width l in LAN; (b)Runtime for different parallel batch size bs in the LAN setting with $l = 32$; (c)Runtime for different bit-width l in WAN; (d)Runtime for different parallel batch size bs in the WAN setting with $l = 32$.

TABLE IV
COMPARISON OF COMMUNICATION WITH PRIOR WORKS FOR
PRIVATE ReLU FUNCTION

Works	Comm. (bits)	Rounds	Result Privacy
MiniONN [38]	$18\lambda\eta - 6\lambda$	2	Strong
Huang et al. [22]	$10l - 4$	$l + 1$	Weak
Li et al. [21]	$8\lambda l - 4\lambda$	2	Strong
CryptFlow2 [20]	$\lambda l + 18l$	$\log l + 2$	Strong
This work	$13l$	4	Strong

The parameter l denotes the length of an integer in the ring $\mathbb{Z}_{2^l}$, $\eta = \lceil \log n \rceil$ for a ring $\mathbb{Z}_n$, while λ is the security parameter and typically 128.

Li et al.'s scheme replaces the MaxPool with AvgPool. Figs. 9 and 10 show the experimental results in the LAN and WAN settings for Network-II and Network-III respectively.

On the MNIST dataset with Network-II, when compared with MiniONN, Li et al.'s scheme, PCNN_{CEC} respectively gains $26.38\times$ and $17.59\times$ improvements on runtime in the LAN setting, and $4.39\times$ and $2.77\times$ improvements on runtime in the WAN setting. On the CIFAR-10 dataset with Network-III, when compared with MiniONN and Li et al.'s scheme, PCNN_{CEC} respectively gains $2.26\times$ and $2.74\times$ improvements on runtime

TABLE V
COMPARISON OF COMMUNICATION WITH PRIOR WORKS FOR PRIVATE
MAXPOOL ALGORITHM

Works	Comm. (bits)	Rounds
MiniONN [38]	$(k-1)(18\lambda\eta - 6\lambda)$	2
Huang et al. [22]	$(k-1) \cdot (10l - 4)$	$(l+1) \cdot \lceil \log k \rceil$
Li et al. [21]	—	—
CryptFlow2 [20]	$(k-1) \cdot (\lambda l + 14l)$	$\log l \cdot \lceil \log k \rceil$
This work	$12(k-1)l$	$4\lceil \log k \rceil$

The parameter l denotes the length of an integer in the ring $\mathbb{Z}_{2^l}$, k denote the size of a pool filter, while λ is the security parameter and typically 128.

in the LAN setting, and $1.29\times$ and $1.47\times$ improvements on runtime in the WAN setting. Therefore, PCNN_{CEC} can be performed well in both LAN and WAN settings.

VIII. CONCLUSION

For Industrial Internet of Things, accuracy, privacy preservation and low latency are the key issues in deploying the inference of convolutional neural network architecture. To protect the privacy of data and model, previous works usually applied the homomorphic encryption, garbled circuits and secret sharing to realize private inference. However, most of them cannot achieve a good balance among the three requirements.

TABLE VI
PERFORMANCE OF EACH LAYER ON THE MNIST DATASET

Network I			Network II		
Layers	Latency (s)		Layers	Latency (s)	
	LAN	WAN		LAN	WAN
1.Convolution	1.603×10^{-2}	0.042	1.Convolution	0.082	0.17
2.Square	7.815×10^{-4}	0.03	2.ReLU	0.038	0.189
3.AvgPool	8.449×10^{-8}	9.194×10^{-8}	3.MaxPool	0.027	0.383
4.Square	3.11×10^{-4}	0.029	4.Convolution	0.008	0.045
5.Full-connected	8.604×10^{-4}	0.032	5.ReLU	0.005	0.124
-	-	-	6.MaxPool	0.005	0.238
-	-	-	7.Full-connected	0.008	0.512
-	-	-	8.ReLU	0.001	0.117
-	-	-	9.Full-connected	0.0007	0.03
Total	0.018	0.133	Total	0.175	1.808

TABLE VII
PERFORMANCE OF EACH LAYER ON THE CIFAR-10 DATASET IN NETWORK III

Layers	Latency (s)		Layers	Latency (s)	
	LAN	WAN		LAN	WAN
1.Convolution	0.54	1.109	10.AvgPool	0.00001	0.00001
2.ReLU	0.261	0.636	11.Convolution	0.72	1.469
3.Convolution	11.52	46.109	12.ReLU	0.017	0.149
4.ReLU	0.287	0.812	13.Convolution	0.08	0.189
5.AvgPool	0.00001	0.00001	14.ReLU	0.035	0.123
6.Convolution	2.88	5.789	15.Convolution	0.02	0.113
7.ReLU	0.067	0.346	16.ReLU	0.007	0.215
8.Convolution	2.912	6.011	17.Full-connected	0.004	0.076
9.ReLU	0.089	0.346	Total	19.439	63.492

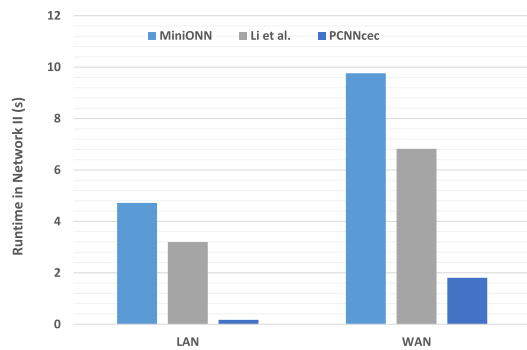


Fig. 9. Comparison of runtime for Network II.

Therefore, by only applying arithmetic secret sharing, we designed a new framework PCNN_{CEC} based on the cloud-edge-client collaboration that the private inference is moved to the edge. And we delegated the task of triplets generation to the cloud to speed up the offline phase but not sacrifice security. Then, we proposed a new two-party comparison protocol to realize private ReLU and maximum pooling. As there is no

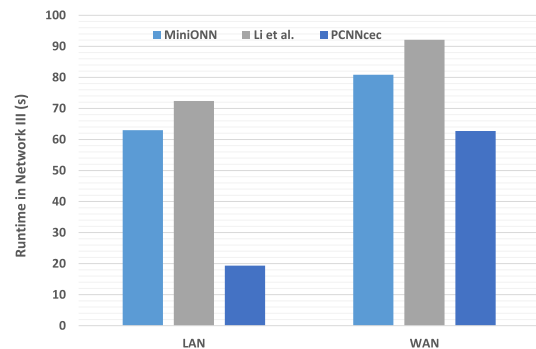


Fig. 10. Comparison of runtime for Network III.

approximation in private activation computation, PCNN_{CEC} maintains the same prediction accuracy as that in traditional centralized inference. The experimental results further demonstrated that PCNN_{CEC} can achieve a better tradeoff among accuracy, privacy preservation and low latency when compared with other recently proposed schemes in the same security level.

REFERENCES

- [1] B. Chen, J. Wan, Y. Lan, M. Imran, D. Li, and N. Guizani, "Improving cognitive ability of edge intelligent IIoT through machine learning," *IEEE Netw.*, vol. 33, no. 5, pp. 61–67, Sep./Oct. 2019.
- [2] P. C. M. Arachchige, P. Bertok, I. Khalil, D. Liu, S. Camtepe, and M. Atiquzzaman, "A trustworthy privacy preserving framework for machine learning in industrial IoT systems," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 6092–6102, Sep. 2020.
- [3] H. Yao, P. Gao, P. Zhang, J. Wang, C. Jiang, and L. Lu, "Hybrid intrusion detection system for edge-based IIoT relying on machine-learning-aided detection," *IEEE Netw.*, vol. 33, no. 5, pp. 75–81, Sep./Oct. 2019.
- [4] W. Dou, X. Zhao, X. Yin, H. Wang, Y. Luo, and L. Qi, "Edge computing-enabled deep learning for real-time video optimization in IIoT," *IEEE Trans. Ind. Informat.*, vol. 17, no. 4, pp. 2842–2851, Apr. 2021.
- [5] R. V. LOON, "Edge-ifying machine learning for industrial IoT," Jun. 2022. [Online]. Available: <https://www.thedigitaltransformationpeople.com/channels/enabling-technologies/edge-ifying-machine-learning-industrial-iiot/>.
- [6] N. John and S. Sam, "Provably secure data sharing approach for personal health records in cloud storage using session password, data access key, and circular interpolation," *Int. J. Semantic Web Inf. Syst.*, vol. 17, no. 4, pp. 76–98, 2021.
- [7] A. Tewari and B. B. Gupta, "Secure timestamp-based mutual authentication protocol for IoT devices using RFID tags," *Int. J. Semantic Web Inf. Syst.*, vol. 16, no. 3, pp. 20–34, 2020.
- [8] T. Wang, Y. Yao, Y. Chen, M. Zhang, F. Tao, and H. Snoussi, "Auto-sorting system toward smart factory based on deep learning for image segmentation," *IEEE Sensors J.*, vol. 18, no. 20, pp. 8493–8501, Oct. 2018.
- [9] F. Li, C. Tan, F. Dong, and J. Jia, "V-Net deep imaging method for electrical resistance tomography," *IEEE Sensors J.*, vol. 20, no. 12, pp. 6460–6469, Jun. 2020.
- [10] R. Taheri, M. Shojafar, M. Alazab, and R. Tafazolli, "Fed-IIoT: A robust federated malware detection architecture in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 8442–8452, Dec. 2021.
- [11] Y. Liu *et al.*, "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet of Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.
- [12] P. Goswami, A. Mukherjee, M. Maiti, S. K. S. Tyagi, and L. Yang, "A neural network based optimal resource allocation method for secure IIoT network," *IEEE Internet of Things J.*, vol. 9, no. 4, pp. 2538–2544, Feb. 2022.
- [13] Y. Wang, K. Hong, J. Zou, T. Peng, and H. Yang, "A CNN-based visual sorting system with cloud-edge computing for flexible manufacturing systems," *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4726–4735, Jul. 2020.
- [14] G. A. Kaissis, M. R. Makowski, D. Rückert, and R. F. Braren, "Secure, privacy-preserving and federated machine learning in medical imaging," *Nat. Mach. Intell.*, vol. 2, no. 6, pp. 305–311, 2020.
- [15] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction APIs," in *Proc. 25th Secur. Symp.*, 2016, pp. 601–618.
- [16] X. Gong, Q. Wang, Y. Chen, W. Yang, and X. Jiang, "Model extraction attacks and defenses on cloud-based machine learning models," *IEEE Commun. Mag.*, vol. 58, no. 12, pp. 83–89, Jan. 2021.
- [17] B. D. Rouhani, M. S. Riazi, and F. Koushanfar, "Deepsecure: Scalable provably-secure deep learning," in *Proc. 55th Annu. Des. Automat. Conf.*, 2018, pp. 1–6.
- [18] M. S. Riazi, M. Samragh, H. Chen, K. Laine, K. Lauter, and F. Koushanfar, "{XONN}: Xnor-based oblivious deep neural network inference," in *Proc. 28th Secur. Symp.*, 2019, pp. 1501–1518.
- [19] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th Secur. Symp.*, 2020, pp. 2505–2522.
- [20] D. Rathee *et al.*, "Cryptflow2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [21] M. Li, S. S. Chow, S. Hu, Y. Yan, S. Chao, and Q. Wang, "Optimizing privacy-preserving outsourced convolutional neural network predictions," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1592–1604, May/Jun. 2022.
- [22] K. Huang, X. Liu, S. Fu, D. Guo, and M. Xu, "A lightweight privacy-preserving CNN feature extraction framework for mobile sensing," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1441–1455, May/Jun. 2021.
- [23] A. C.-C. Yao, "How to generate and exchange secrets," in *Proc. 27th Annu. Symp. Found. Comput. Sci.*, 1986, pp. 162–167.
- [24] D. Demmler, T. Schneider, and M. Zohner, "ABY-A framework for efficient mixed-protocol secure two-party computation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, pp. 1–15, 2015.
- [25] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2.0: Improved mixed-protocol secure two-party computation," in *Proc. 30th Secur. Symp.*, 2021, pp. 2165–2182.
- [26] P. Mohassel and P. Rindal, "Aby3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 35–52.
- [27] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Proc. Annu. Int. Cryptol. Conf.*, 1991, pp. 420–432.
- [28] E. Hesamifard, H. Takabi, M. Ghasemi, and R. N. Wright, "Privacy-preserving machine learning as a service," *Proc. Priv. Enhancing Technol.*, vol. 2018, no. 3, pp. 123–142, 2018.
- [29] B. B. Gupta *et al.*, "Blockchain-assisted secure fine-grained searchable encryption for a cloud-based healthcare cyber-physical system," *IEEE/CAA J. Automatica Sinica*, vol. 8, no. 12, pp. 1877–1890, Dec. 2021.
- [30] J. W. Bos, K. Lauter, and M. Naehrig, "Private predictive analysis on encrypting medical data," *J. Biomed. Informat.*, vol. 50, pp. 234–243, 2014.
- [31] X. Sun, P. Zhang, J. K. Liu, J. Yu, and W. Xie, "Private machine learning classification based on fully homomorphic encryption," *IEEE Trans. Emerg. Top. Comput.*, vol. 8, no. 2, pp. 352–364, Apr.–Jun. 2020.
- [32] S. Park, J. Lee, J. H. Cheon, J. Lee, J. Kim, and J. Byun, "Security-preserving support vector machine with fully homomorphic encryption," in *Proc. Workshop Artif. Intell. Saf.*, 2019.
- [33] L. Liu, R. Chen, X. Liu, J. Su, and L. Qiao, "Towards practical privacy-preserving decision tree training and evaluation in the cloud," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 2914–2929, 2020.
- [34] J. Xiong, R. Bi, Y. Tian, X. Liu, and D. Wu, "Towards lightweight, privacy-preserving cooperative object classification for connected autonomous vehicles," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2787–2801, Feb. 2022.
- [35] J. Wang, L. Wu, H. Wang, K.-K. R. Choo, and D. He, "An efficient and privacy-preserving outsourced support vector machine training for Internet of medical Things," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 458–473, Jan. 2021.
- [36] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
- [37] M. Naor and B. Pinkas, "Computationally secure oblivious transfer," *J. Cryptol.*, vol. 18, no. 1, pp. 1–35, 2005.
- [38] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minion transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [39] N. Chandran, D. Gupta, A. Rastogi, R. Sharma, and S. Tripathi, "EZPC: Programmable and efficient secure two-party computation for machine learning," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2019, pp. 496–511.
- [40] S. Albawi, T. A. Mohammed, and S. Al-Zawi, "Understanding of a convolutional neural network," in *Proc. Int. Conf. Eng. Technol.*, 2017, pp. 1–6.
- [41] L. Xiong, W. Zhou, Z. Xia, Q. Gu, and J. Weng, "Efficient privacy-preserving computation based on additive secret sharing," 2020, *arXiv:2009.05356*.
- [42] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd IEEE Symp. Found. Comput. Sci.*, 2001, pp. 136–145.
- [43] R. Canetti, A. Cohen, and Y. Lindell, "A simpler variant of universally composable security for standard multiparty computation," in *Proc. Annu. Cryptol. Conf.*, 2015, pp. 3–22.
- [44] Q. Feng, D. He, Z. Liu, H. Wang, and K.-K. R. Choo, "SecureNLP: A system for multi-party privacy-preserving natural language processing," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3709–3721, 2020.
- [45] Y. LeCun, C. Cortes, and C. Burges, "The MNIST handwritten digit database," 2022. [Online]. Available: <http://yann.lecun.com/exdb/mnist>.
- [46] A. Krizhevsky *et al.*, "Learning multiple layers of features from tiny images," Master's thesis, Univ. Toronto, Toronto, ON, Canada, 2009.
- [47] X. Wang, A. J. Malozemoff, and J. Katz, "EMP-toolkit: Efficient multiparty computation toolkit," 2022. [Online]. Available: <https://github.com/emp-toolkit>.



Jing Wang received the Ph.D. degree in computer science from Wuhan University, Wuhan, China, in 2021. She is currently a Lecturer with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China. Her main research interests include cryptography and secure multi-party computation, in particular, secure cloud storage, and cryptographic protocols.



Debiao He (Member, IEEE) received the Ph.D. degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China, in 2009. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China. His main research interests include cryptography and information security, in particular, cryptographic protocols. He has authored or coauthored more than 100 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON DEPENDABLE

AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION SECURITY AND FORENSIC, and Usenix Security Symposium. He was the recipient of the 2018 IEEE Systems Journal Best Paper Award and the 2019 IET Information Security Best Paper Award. His work has been cited more than 10000 times at Google Scholar. He is also with the editorial board of several international journals, such as *Journal of Information Security and Applications*, *Frontiers of Computer Science*, and *Human-centric Computing & Information Sciences*.



Aniello Castiglione (Member, IEEE) received the Ph.D. degree in computer science from the University of Salerno, Fisciano, Italy. He is currently a Professor with the University of Naples, Naples, Italy. He has authored or coauthored more than 240 papers in international journals and conferences. Considering his journal papers, more than 85 of them are ranked Q1 in Scopus/Scimago classification and more than 70 of them are ranked Q1 in the Clarivate Analytics/ISI-WoS classification. The international academic profile of Dr. Castiglione is spread among his 86 international

co-authors who belong to 75 different institutions located in 18 countries. He served in the organization (mainly as the Program Chair and a TPC member) in about 250 international conferences (some of them are ranked A+/A/A- in the CORE, LiveSHINE and Microsoft Academic international classifications). He is currently the Editor-in-Chief of the Special Issues for the *Journal of Ambient Intelligence and Humanized Computing* (Springer). One of his papers (published in the IEEE TDSC) was selected as Featured Article in the IEEE Cybersecurity Initiative in 2014. In October 2020 and October 2021, he has been included into the ranking of the top 100,000 scientists for the years 2019 and 2020. His current research interests include information forensics, digital forensics, security and privacy on cloud, communication networks, applied cryptography, and sustainable computing.



Brij B. Gupta (Senior Member, IEEE) received the Ph.D. degree from the Indian Institute of Technology (IIT) Roorkee, Roorkee, India. In more than 16 years of his professional experience, he has authored or coauthored more than 400 papers in journals/conferences, including 25 books and eight Patents with more than 14000 citations. He was the recipient of numerous national and international awards, including Canadian Commonwealth Scholarship in 2009, Faculty Research Fellowship Award in 2017, MeitY, GoI, IEEE GCCE outstanding and WIE paper awards

and Best Faculty Award between 2018 and 2019, at NIT Kurukshetra. He is also selected in the 2021 and 2020 Stanford University's ranking of the world's top 2% scientists. He is/was also a visiting/adjunct professor with several universities worldwide. His research interests include information security, cyber physical systems, cloud computing, blockchain technologies, intrusion detection, AI, and social media and networking. He is also selected as a 2021 Distinguished Lecturer in IEEE CTSoc. Prof. Gupta is also a Member-in-Large, Board of Governors, IEEE Consumer Technology Society during 2022–2024. Prof. Gupta is also leading IJSWIS, IJSSCI and IJCAC as the Editor-in-Chief. He is also the Lead-Editor of a Book Series with CRC and IET press. He was also TPC members in more than 150 international conferences. Dr Gupta is/was also an Associate/Guest Editor of various journals and transactions. Prof. Gupta is currently the Director, International Center for AI and Cyber Security Research, Incubation and Innovations, and Full Professor with the Department of Computer Science and Information Engineering, Asia University, Taichung, Taiwan.



Marimuthu Karuppiyah (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from VIT University, Vellore, India, in 2015. He was an Associate Professor with VIT University, Vellore, India. He is currently a Professor with the Department of Computing Science and Engineering, SRM Institute of Science and Technology, Delhi-NCR Campus, India. He has authored or coauthored more than 30 research articles in SCI indexed journals. He has also authored or coauthored more than 40 research articles in SCOPUS indexed

journals and international conferences. His research interests include cryptography and wireless network security, in particular, authentication and encryption schemes. He is a Life Member of the Cryptology Research Society of India and the Computer Society of India, and a Member of ACM.



Libing Wu received the B.S. and M.S. degrees in computer science from Central China Normal University, Wuhan, China, in 1994 and 2001, respectively, and the Ph.D. degree in computer science from Wuhan University, Wuhan, China, in 2006. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. His research interests include distributed computing, trusted software, and wireless sensor networks. He is also a Member of China Computer Federation.